ST. EDWARD'S UNIVERSITY

EXPLORING VIBRATION SIMULATED BRAILLE FOR ACCESSIBLE MOBILE MESSAGING

by

Swapnil Puri

A Capstone Project
Submitted to
Dr. Bilal Shebaro

St. Edward's University

April 2026

© Copyright 2026 by Swapnil Puri

All Rights Reserved

Abstract

In the contemporary digital landscape, electronic devices serve as the primary gateway to information. However, accessibility remains inequitable for individuals who are blind or deafblind. Most smartphones lack native tactile braille output and instead require specialized external hardware that is often expensive and impractical. This project presents Vibcom, an accessible iOS messaging application built using Apple's SwiftUI framework and supported by Google's Firebase for backend authentication and cloud data management. Vibcom explores the potential use of haptic feedback to simulate the six-dot structure of a braille cell through structured vibration patterns composed of strong and weak pulses that could represent raised and flat dots. This approach aims to enable users to perceive textual information through touch. Users can compose messages through gesture-based interactions that replicate braille input, facilitating nonvisual communication. To support comprehensive accessibility, Vibcom integrates three output modalities including haptic, auditory, and visual text, as well as three input modalities including gesture-based braille simulation, letter drawing, and standard text entry. Gesture driven navigation further reduces reliance on fixed interface elements. By implementing a multimodal communication framework, Vibcom advances inclusive design and demonstrates the potential of vibration-based braille simulation as a scalable and integrated approach to accessible mobile messaging.

Keywords

Accessible communication, haptic feedback, vibration-based braille; multimodal interaction; assistive technology, mobile accessibility, inclusive design, gesture-based input, tactile information systems, deaf blind communication.

Acknowledgements

The author thanks Dr. Bilal Shebaro for guidance and technical feedback, as well as Dr. Cory Lock for academic mentorship. The author also acknowledges Dr. Emma Woelk for supporting an environment of academic excellence. Finally, the author thanks their family for their continued support.

Table of Contents

1. Introduction 1
2. Background and Related Work 4
3. System Requirements 7
   3.1 Software Requirements 7
      3.1.1 Functional Requirements 7
      3.1.2 Non-Functional Requirements 9
   3.2 Accessibility Requirements 10
   3.3 User Requirements 11
4. System Design 13
   4.1 System Architecture Overview 13
   4.2 Vibcom Design 14
   4.3 Vibconnect Design 16
   4.4 Data Flow and Communication 17
   4.5 Interaction and Navigation Design 17
5. Accessibility Design Principles 19
   5.1 Gesture-Based Interaction 19
   5.2 Braille-Inspired Input Methods 19
   5.3 Multimodal Feedback Systems 20
   5.4 Non-Visual Navigation and Usability 21
6. Implementation 22
   6.1 Tools and Technologies 22
   6.2 System Integration and Data Management 23
   6.3 Key Features of Vibcom 23
   6.4 Key Features of Vibconnect 24
   6.5 Development Challenges 25
7. Evaluation 26
   7.1 Functional Testing 26
   7.2 Accessibility Validation 27
   7.3 Usability Assessment 28
8. Discussion 30
   8.1 System Strengths 30
   8.2 Design Tradeoffs and Challenges 31
   8.3 Implications for Accessible Application Design 31
9. Limitations 33
10. Conclusion 34
11. References 35
12. Appendices 38
   12.1 System and Workflow Diagrams 38
   12.2 User Interface Screenshots 46

1. Introduction

In the modern digital era, mobile devices have become one of the primary means through which people communicate, access information, and maintain social connections. Despite the widespread use of these technologies, digital accessibility remains uneven. Many mobile applications continue to rely on visual layouts, small touch targets, and interface conventions that assume users can readily see and interpret the screen. For individuals who are blind or visually impaired, these assumptions can create major barriers to independent and efficient communication.

One established method of nonvisual information access is braille. Braille remains closely tied to literacy, independence, and psychological well being among visually impaired individuals (Schroeder, 1996; Silverman & Bell, 2018). However, extending braille into digital environments presents significant challenges. Refreshable braille displays provide tactile access to electronic content, but they are often prohibitively expensive and require specialized hardware. For many users, the cost of these devices places them beyond practical reach, with common models ranging from several thousand to tens of thousands of dollars (Dolphin et al., 2024). As a result, many visually impaired users rely primarily on audio based tools such as screen readers and text to speech systems. While these technologies are valuable, they do not fully reproduce the tactile benefits associated with braille based interaction.

In response to this limitation, researchers have explored alternative methods of delivering tactile information through widely available consumer devices. Among the most promising approaches is the use of vibration and haptic feedback on mobile phones. Prior work has shown that users can interpret vibrational patterns to recognize information such as environmental cues and braille characters (Khusro et al., 2022; Jayant et al., 2010). One notable example is V Braille,

which uses touchscreen interaction and vibration to simulate braille patterns on a mobile device (Jayant et al., 2010). These studies demonstrate that vibration based tactile interaction can offer meaningful accessibility benefits without requiring dedicated braille hardware.

Although this prior work is important, most existing systems remain limited in scope. Many are experimental prototypes designed to test isolated tasks such as letter recognition or basic braille input. They do not typically extend into complete applications that support sustained communication, contact management, and social interaction in a practical mobile environment. At the same time, mainstream messaging and social discovery applications remain heavily visual in both structure and interaction. Users are commonly expected to scan conversation lists, interpret status indicators, type on conventional keyboards, and navigate visually dense screens. These expectations make both communication and social connection more difficult for users who depend on nonvisual interaction.

This project addresses that gap through the design and development of two interconnected mobile applications, Vibcom and Vibconnect. Vibcom is an accessibility focused messaging application that supports communication through gesture based interaction, braille inspired input, and multimodal feedback using audio and tone based cues. Vibconnect serves as a companion application that supports nearby user discovery and contact integration, allowing users to establish new connections through an interface designed with accessibility in mind. Together, the two applications form a unified communication ecosystem that extends accessibility beyond message exchange alone and into the broader process of social connection.

The significance of this work lies in its accessibility first approach. Rather than treating accessibility as a supplementary layer added onto a conventional mobile design, Vibcom and Vibconnect were developed around nonvisual interaction as a central design principle. This paper

presents the design, implementation, and evaluation of these applications as a complete accessibility oriented communication system. In doing so, it shows how mobile technologies can be rethought to better support blind and visually impaired users through integrated, practical, and socially relevant design.

2. Background and Related Work

Braille has long served as a fundamental method for reading and writing among individuals who are blind or visually impaired. Its importance extends beyond basic communication, as it is closely tied to literacy, independence, and psychological well being. Prior research has shown that braille proficiency contributes to a stronger sense of competence and self confidence among visually impaired adults (Schroeder, 1996). More recent studies have reinforced this connection, showing that braille literacy is associated with improved well being and long term educational outcomes (Silverman and Bell, 2018). Despite its importance, integrating braille into modern digital systems remains a significant challenge.

One widely adopted solution is the use of refreshable braille displays, which convert digital text into tactile braille characters through dynamically raised pins. These devices allow users to access digital information in real time and are compatible with many existing systems. However, their adoption is limited by several practical constraints. Cost remains the most significant barrier, with devices often priced between several thousand and tens of thousands of dollars (Dolphin et al., 2024). In addition, users have reported concerns related to portability, device weight, and operational noise (Martinez and Kamei Hannan, 2025). While refreshable braille displays provide an effective tactile interface, these limitations restrict their accessibility and widespread use.

In response to these challenges, researchers have explored alternative methods for delivering tactile information without requiring specialized hardware. One promising direction involves the use of vibration and haptic feedback. Mobile devices, which are widely available and already equipped with vibration motors, offer a practical platform for this approach. Studies have shown that users can interpret distinct vibration patterns to convey meaningful information.

For example, vibration based systems have been used to assist blind users in navigating indoor environments, with high levels of accuracy in interpreting directional cues (Khusro et al., 2022). Similarly, wearable systems that map braille dots to vibrations on different parts of the hand have demonstrated that users can successfully read and transmit simple words through tactile feedback (Ozioko et al., 2020).

Building on this idea, researchers have developed mobile based systems that simulate braille directly on touchscreen devices. One of the earliest and most notable examples is VBraille, which divides the touchscreen into regions corresponding to the six dots of a braille cell. When users interact with these regions, the device produces vibration patterns that represent raised dots, allowing users to interpret characters through touch (Jayant et al., 2010). Studies evaluating VBraille reported high levels of accuracy in letter recognition, demonstrating that vibration based braille simulation is a viable method for tactile interaction on mobile devices.

Subsequent research has explored alternative interaction techniques for braille input and output. Some systems use on screen braille keyboards that map each dot of a braille cell to a specific touch area, providing vibration or audio feedback when pressed (Voutsakelis et al., 2024). Other approaches have investigated stylus based interaction, where users hold a stylus on the screen and receive sequential vibrations representing each dot in a braille character (Rantala et al., 2009). These methods aim to improve input efficiency and accuracy, while reducing the need for precise visual targeting.

Despite these advances, existing implementations of vibration simulated braille face several limitations. Many systems are designed primarily for experimental evaluation rather than practical use. As a result, they often focus on isolated tasks such as recognizing individual letters or entering short sequences of text. They do not typically support continuous interaction with

meaningful content, nor do they integrate into fully functional applications that support communication over time. In addition, usability challenges remain. Users have reported issues such as accidental inputs, insufficient feedback clarity, and discomfort with certain interaction methods (Voutsakelis et al., 2024; Rantala et al., 2009).

At the same time, research on accessible mobile applications has shown that many mainstream systems continue to prioritize visual design patterns. Messaging platforms and social applications often rely on visual navigation, complex layouts, and standard text input methods. While accessibility features such as screen readers provide support, they are frequently implemented as secondary layers rather than as core design principles. This can result in interactions that are less efficient and less intuitive for users who depend on nonvisual input.

Taken together, this body of work highlights both the potential and the limitations of current approaches. Vibration based braille simulation offers a promising method for delivering tactile information on widely available devices. However, there remains a gap between experimental prototypes and fully realized applications that support real world communication and social interaction. This project builds on prior research by extending vibration based and multimodal interaction into a complete mobile communication ecosystem, integrating accessible messaging with accessible methods for discovering and connecting with other users.

3. System Requirements

The development of Vibcom and Vibconnect was guided by a requirements-driven design process in which accessibility was treated as a primary system constraint rather than a secondary enhancement. Because the project is intended to support communication for users who may not rely on vision, the requirements extend beyond ordinary mobile messaging functionality. In addition to authentication, message exchange, and persistence, the system must support nonvisual navigation, multimodal feedback, contact synchronization, nearby discovery, and reliable coordination between local and cloud-backed state. For clarity, the requirements are divided into software requirements, accessibility requirements, and user requirements. This distinction is important because the success of the system cannot be measured only by whether messages can be transmitted. It must also be measured by whether those tasks can be completed independently, consistently, and confidently by the intended users.

3.1 Software Requirements

The software requirements define the capabilities and quality expectations of the overall system. These requirements include both what the applications must do and how they must behave in practice under normal usage conditions.

3.1.1 Functional Requirements

At a core level, Vibcom must support secure and persistent user identity. Users are required to authenticate through phone-number verification, creating an account that can be used consistently across sessions and devices. This identity must be suitable not only for messaging, but also for contact matching, nearby discovery, and presence updates. In the implemented

system, this requirement is fulfilled through Firebase Authentication. The messaging layer must support one-to-one conversation creation, message transmission, message receipt, and persistent message history. Users must be able to reopen the application and recover prior conversations without losing data. This requirement is especially important because an accessibility-oriented communication tool would become impractical if it could not preserve conversational continuity. To satisfy this requirement, Vibcom combines real-time Firestore synchronization with local persistence using SwiftData.

A further functional requirement is real-time synchronization. When a user sends a message, it should become visible on the recipient device without requiring manual refresh. Likewise, conversation summaries and presence indicators should update dynamically so that the application behaves as a live communication platform rather than a static message archive. This requirement influenced the use of cloud listeners and real-time coordination within the system.

Another major functional requirement concerns text entry. Vibcom must support more than one method of message composition. In the implemented system, users can compose text through standard keyboard input, gesture-based braille-style entry, and drawing-based character entry. These input methods are required because different users may prefer different strategies depending on familiarity, dexterity, and context of use.

The system must also support device contact integration. Vibcom is required to identify contacts who are already registered users and generate communication pathways accordingly. If a relevant contact is added and corresponds to a registered user, a chat should become available. Conversely, if that contact is removed, the corresponding contact-derived chat should not remain visible. This requirement ensures that the messaging environment reflects the user’s actual contact network rather than maintaining stale or misleading communication relationships.

Vibconnect introduces additional functional requirements. It must support nearby user discovery through an accessible interaction model, allow users to accept or skip candidates through simple gestures, avoid surfacing the current user as a discovery candidate, and prevent skipped entries from repeatedly reappearing in normal usage. It must also support direct nearby exchange between devices, allowing a user to acquire a contact through local exchange and then continue communication through Vibcom.

Finally, the system must provide multimodal feedback. Messages, navigation state, confirmations, and error conditions must be available through nonvisual feedback channels such as speech and tone-based cues, while still preserving visible output where appropriate.

3.1.2 Non-Functional Requirements

A major non-functional requirement is responsiveness. Since Vibcom and Vibconnect depend heavily on gesture interaction, focus movement, and immediate confirmation, delays in updating UI state or feedback can significantly degrade usability. Switching between modes, moving focus, receiving a live message, or confirming an action must occur with minimal latency to preserve user orientation. Reliability is equally important. Messages must persist correctly, chat state must remain stable across sessions, and local and cloud-backed data must stay synchronized. Failures such as disappearing messages, duplicate content, stale status indicators, or mismatched contact state would reduce trust in the application and undermine its suitability as an accessible tool. Cross-device and cross-layout consistency is another important requirement. The applications are intended to function on both iPhone and iPad, requiring that layouts, launch screens, onboarding flows, and gesture-sensitive views adapt appropriately to different viewport sizes without losing clarity or interaction consistency. Security and privacy are

also required qualities of the system. Since the applications handle phone numbers, contacts, location-derived nearby-discovery information, and real-time communication data, user state must be managed securely. Authentication, permission-aware behavior, and sensible data separation are therefore necessary at both the interface and backend levels. A final non-functional requirement is interface quality. Because the system is intended for serious academic presentation and practical use, it must feel polished, stable, and coherent. In an accessibility-centered system, visual and interaction polish is not merely decorative. Inconsistent layouts, weak state continuity, or confusing feedback timing can directly harm accessibility and therefore must be treated as substantive quality issues.

3.2 Accessibility Requirements

Accessibility is the primary design requirement of the project. The most important accessibility requirement is that all core tasks must be achievable without reliance on vision. Users must be able to authenticate, navigate, compose messages, interpret feedback, and discover new users without depending on visual scanning of the interface. A second requirement is interaction consistency. Vibcom and Vibconnect must not require unrelated or contradictory control patterns. Common actions should follow repeated logic wherever possible, allowing users to form stable expectations and transfer learned interaction habits between applications. The system must also provide redundant feedback. No important action should be communicated exclusively through a visual change. Instead, confirmations, errors, and state transitions should be reinforced through accessible output such as spoken feedback or tone-based braille-style cueing. The interface must avoid accessibility-hostile layouts. Small, densely packed touch targets, visually overloaded screens, and hidden critical controls are inappropriate for the

intended users. Accordingly, the system must use large action regions, clear hierarchy, predictable structure, and gesture-based alternatives to visually located buttons. A final accessibility requirement is flexibility. The system cannot assume that all users are braille-literate, nor can it assume that the same feedback modality will be optimal in all situations. As a result, the application must support multiple input and output pathways while maintaining a coherent overall interaction model.

3.3 User Requirements

From the user perspective, the system must support independent communication. Users should be able to send, receive, and manage messages without needing another person to visually operate the interface or confirm the result of actions. Users also require predictability. Gesture consequences must remain stable, feedback must be timely, and interface state should not change in ways that are difficult to interpret nonvisually. Predictability is particularly important in accessible systems because users cannot easily rely on visual cross-checking when uncertain. Ease of learning is another user requirement. The project includes nonstandard interaction techniques such as braille-style gesture entry and swipe-based discovery. Therefore, the system must remain learnable through repetition and consistency rather than through a large, highly context-dependent gesture vocabulary. Efficiency is also required. Common tasks such as opening a highlighted chat, entering a message, sending that message, or accepting a nearby user should not involve excessive intermediary steps. A technically accessible system can still be functionally weak if ordinary actions require too much effort. Finally, users require continuity between discovery and communication. Vibconnect only becomes meaningful as part of the broader ecosystem if accepted users can subsequently become communication partners within

Vibcom. The system must therefore support a practical flow from discovery to sustained messaging.

4. System Design

The design of Vibcom and Vibconnect reflects the goal of creating an integrated accessibility-first communication ecosystem rather than a single isolated application. Vibcom serves as the primary messaging platform, while Vibconnect extends the system by supporting nearby user discovery and contact acquisition. Although the two applications differ in immediate purpose, they share a common identity model, overlapping cloud data, and a consistent accessibility philosophy. The system design is shaped by three broad priorities: accessibility-centered interaction, modular architecture, and dependable synchronization between local and cloud-backed state. Accessibility-centered interaction ensures that the applications are usable without sight. Modular architecture separates identity, messaging, contact handling, nearby discovery, and feedback systems into understandable subsystems. Dependable synchronization ensures that the applications behave consistently even as they combine local persistence with live cloud updates.

4.1 System Architecture Overview

The overall architecture follows a client-cloud model. The client side consists of two native iOS applications built with SwiftUI. The backend is centered on Firebase Authentication and Cloud Firestore. Device-level frameworks such as Contacts, Core Location, and Multipeer Connectivity provide additional capabilities that support discovery and system integration. Firebase Authentication is used to establish user identity through phone-number verification. This design choice supports a stable and practical identity model, because phone numbers can also be used for contact matching and user discovery. Once authenticated, the user is associated with a Firestore profile that stores relevant metadata such as phone number, presence status, and

nearby-discovery information. Cloud Firestore is used to store user records, message data, conversation summaries, presence information, and nearby-discovery metadata. Firestore was an appropriate choice because it supports snapshot listeners, making it well suited for the real-time communication behavior required by Vibcom. These listeners are used to keep message lists, open chats, and status information synchronized across devices. On the device, Vibcom uses SwiftData as a local persistence layer for users, conversations, and messages. This is a significant architectural decision because it prevents the user experience from depending entirely on cloud availability or view-level listener timing. Local persistence supports continuity, responsiveness, and safer merge behavior between local and remote state. Contacts integration is handled through Apple's Contacts framework. Nearby discovery depends on Core Location and cloud-backed location metadata. Nearby local exchange in Vibconnect is implemented using Multipeer Connectivity. These native frameworks allow the system to support discovery and social integration without building low-level communication mechanisms from scratch. Architecturally, one of the most important decisions is that Vibconnect is not treated as a separate social network. Instead, it operates over the same underlying user ecosystem as Vibcom. In this sense, Vibcom acts as the primary platform and identity source, while Vibconnect functions as an accessible discovery layer.

4.2 Vibcom Design

Vibcom is designed as the central communication application. Its structure differs from conventional visually dense messaging platforms by emphasizing focus, stable navigation logic, and multimodal accessibility feedback. Rather than expecting users to visually browse a crowded list of conversation rows and interface icons, Vibcom organizes interaction around large cards,

highlighted focus, and gesture-accessible transitions. The authentication flow is based on phone-number verification. Once the user is authenticated, local and cloud profile state are created or updated, allowing the system to maintain continuity of identity across sessions. The home screen is structured as a focused chat list. This is a deliberate departure from standard list-scanning patterns. The system maintains a current highlighted element and supports navigation through gestures, reducing the need for visual search and enabling more stable nonvisual orientation. Within a conversation, Vibcom provides multiple text-entry methods. The standard keyboard remains available for users who prefer it, but the application also supports gesture-based braille-style entry and drawing-based character input. These modes are not peripheral experiments; they are integrated into the composition workflow and can be used as practical alternatives depending on user preference and context. Feedback in Vibcom is multimodal. The system combines spoken narration, tone-based braille-style output, and visible text to support message reading, action confirmation, and state awareness. This design ensures that accessibility is distributed across the system rather than confined to a single assistive feature. Contact synchronization is another important part of Vibcom's design. The application reads device contacts, normalizes phone numbers, compares them to registered users in Firestore, and materializes corresponding communication relationships locally. This supports a messaging environment grounded in the user's actual contact network. Finally, Vibcom is responsible for publishing discovery-relevant user state, such as nearby-discovery metadata and discoverability settings, which Vibconnect then consumes for nearby matching.

4.3 Vibconnect Design

Vibconnect is designed to address an adjacent problem to messaging: accessible social discovery. Rather than replicating Vibcom's functionality, it provides a focused environment in which users can find and add nearby people in a gesture-driven, nonvisual way. The main discovery interface presents one candidate at a time through a card-based layout. This design was chosen to reduce complexity and avoid the kind of visually overloaded browsing experience common in mainstream discovery applications. Presenting one nearby user at a time makes the decision structure simple, predictable, and well suited to gesture interaction. The primary actions are directional: swipe right to accept and swipe left to skip. This interaction model keeps decision logic simple and consistent with the broader design principles of the ecosystem. A user does not need to locate and tap visual acceptance or dismissal buttons to proceed. Vibconnect draws nearby candidates from the shared Vibcom userbase, filtered by location-related metadata and discoverability state. Because of this, Vibconnect acts as a companion application within the same overall system rather than as an isolated service with its own identity model. The application also includes a nearby exchange mode using Multipeer Connectivity. This feature allows nearby devices to exchange number-linked identity information directly, creating a second path to social connection that complements the Firestore-based location discovery model. An additional design concern in Vibconnect is exclusion and persistence logic. The current user must not appear in the stack, already-added contacts should not continue to appear as active candidates, and skipped users should not reappear immediately in a repetitive way. These details are essential to preserving a smooth, believable discovery flow.

4.4 Data Flow and Communication

Data flow in the system can be understood across several related pathways. The authentication pathway begins in Vibcom. A user verifies a phone number through Firebase Authentication, and that identity is then reflected in the user's local and cloud profile state. The messaging pathway is centered on Vibcom. When a message is composed and sent, it is represented locally and also transmitted to Firestore. Active listeners then propagate that update to other devices. Local state ensures continuity, while cloud state ensures cross-device synchronization. The contact pathway begins with device contacts. Contacts are normalized locally, matched to registered system users in Firestore, and then materialized into local chat relationships. This same path also supports removal, ensuring that stale contact-derived communication entries do not remain visible when a contact is deleted. The nearby-discovery pathway connects the two applications. Vibcom publishes location-related user metadata and discoverability state, while Vibconnect queries this shared user pool, filters nearby candidates, and allows accepted users to be added into the contact layer. Finally, Vibconnect's nearby-exchange pathway uses Multipeer Connectivity to allow direct local exchange between nearby devices. This creates a second discovery mechanism that operates alongside the cloud-based location filter. These flows are modular but interconnected. Identity, messaging, contacts, presence, and discovery are distinct subsystems, but together they form the wider communication ecosystem.

4.5 Interaction and Navigation Design

Interaction and navigation are based on predictability, repetition, and immediate accessible feedback. Rather than using complex nested interface structures, the system

emphasizes a smaller set of meaningful actions repeated across contexts. In Vibcom, this results in a focused home screen, stable bottom-area composition controls, and simplified mode-driven interaction. In Vibconnect, it results in a card-based discovery model in which the next action is usually a directional swipe rather than a visually located button. A key design objective is that actions produce immediate accessible confirmation. Focus movement, chat activation, mode switching, nearby acceptance, and message sending are all reinforced through accessible feedback channels. This is essential because nonvisual interfaces require a strong sense of state continuity. Another major design choice is cross-application consistency. Although Vibcom and Vibconnect serve different roles, they share design language, gesture logic, and accessibility priorities. This consistency supports usability, reduces relearning, and helps establish the two applications as parts of a unified system.

5. Accessibility Design Principles

The accessibility design of Vibcom and Vibconnect is grounded in the idea that nonvisual interaction should be treated as primary rather than exceptional. Instead of building a conventional messaging and discovery system and then attempting to make it accessible afterward, the project was designed around how a blind or visually impaired user might interact from the start. Four design principles are especially important within this system: gesture-based interaction, braille-inspired input, multimodal feedback, and nonvisual navigation.

5.1 Gesture-Based Interaction

Gesture-based interaction was selected because it reduces dependence on visually locating buttons and supports the development of repeatable motor patterns over time. Rather than requiring users to scan the screen for discrete controls, the system maps actions onto swipes, taps, and structured interaction regions that can be used more consistently without sight. This approach offers clear benefits, but it also requires restraint. If the gesture vocabulary becomes too large or inconsistent, the result may be difficult to learn and error-prone. For this reason, the system favors a limited set of gestures whose meanings remain stable across screens wherever possible. Gesture-based interaction is also valuable beyond strict nonvisual use. It can support faster interaction, reduce clutter, and allow users to operate the application more discreetly or confidently in contexts where visual attention is divided.

5.2 Braille-Inspired Input Methods

Braille remains central to literacy and communication for many blind users, making braille-inspired interaction a natural design direction for accessible communication software.

However, the project does not attempt to recreate a physical refreshable braille display on a general-purpose mobile device. Instead, it adapts braille logic into mobile-friendly input and output patterns that can operate on commodity hardware. In Vibcom, braille-inspired interaction is represented through gesture-based entry patterns and tone- or vibration-style cue structures. This allows the system to preserve the conceptual value of braille while acknowledging the practical limitations of a touchscreen device. Importantly, the project does not assume universal braille literacy. A user who is blind but not braille-fluent must still be able to use the application effectively. This is why braille-inspired input is implemented as a major pathway rather than the exclusive pathway.

5.3 Multimodal Feedback Systems

No single feedback channel is sufficient for an application like Vibcom. Speech is expressive but can be slow or socially intrusive. Tone-based cueing can be fast and compact, but may be ambiguous without context. Visible text remains important for users with partial sight or for mixed-use contexts. The system therefore adopts a multimodal model. In practice, this means that state and content are communicated through a combination of spoken narration, tone-based braille-style output, and visible text. This redundancy improves robustness and user confidence. If one channel is weak in a particular context, another may still convey the necessary information. Multimodal feedback is especially important for confirming system actions. Sending a message, accepting a discovery candidate, switching input modes, or encountering an error all require clear acknowledgment. The use of multiple reinforcing channels helps prevent uncertainty in these moments.

5.4 Non-Visual Navigation and Usability

Nonvisual navigation requires more than gesture support. It requires predictable structure. A user must be able to infer where they are in the system and what the likely consequence of an action will be, even without seeing the interface. To support this, Vibcom and Vibconnect avoid visually dense arrangements of small controls and instead rely on stable layouts, large action regions, and repeated structural patterns. This reduces the spatial and cognitive burden placed on users. In this project, usability and accessibility are not treated as separate categories. Predictability, immediate feedback, consistent number formatting, stable highlighted focus, and reliable live updates are all usability qualities, but they are also accessibility requirements because they support orientation and trust in a nonvisual interaction environment.

6. Implementation

The implementation of Vibcom and Vibconnect involved translating accessibility-driven design principles into a functioning and reasonably scalable iOS system. This required not only interface construction, but also practical solutions for authentication, persistence, real-time synchronization, contact reconciliation, nearby discovery, and feedback coordination.

6.1 Tools and Technologies

Both applications were developed natively for iOS using Swift and SwiftUI. SwiftUI was especially useful because it supports state-driven rendering, rapid iteration, and layout adaptation across multiple device sizes. These characteristics were valuable because many parts of the system, particularly the accessibility-related surfaces, evolved significantly during development. Vibcom uses SwiftData for local persistence. This supports local storage of user, message, and conversation state so that the application remains responsive and stable even as cloud synchronization occurs. Phone-number-based user identity is implemented through Firebase Authentication. Cloud Firestore is used for user profile data, presence state, conversation summaries, messages, and nearby-discovery metadata. Firestore's listener model aligns well with the system's need for live updates and cross-device synchronization. Apple's Contacts framework is used for contact matching and local contact synchronization. Core Location is used to support nearby discovery. Vibconnect's nearby-exchange functionality is implemented using Multipeer Connectivity. Audio and tone-based feedback are coordinated using native iOS mechanisms and custom accessibility playback logic.

6.2 System Integration and Data Management

A major implementation concern was the coexistence of local and cloud state. Vibcom cannot depend entirely on cloud reads because message continuity and perceived reliability are essential. At the same time, it must reflect live remote updates when another user sends a message or changes presence state. The implemented approach therefore uses a hybrid local-cloud model. Local persistence preserves message continuity and supports responsive UI state. Firestore listeners propagate updates across devices. The challenge lies in reconciliation: the system must merge local and cloud state carefully rather than using destructive replacement logic that could cause disappearing, duplicated, or misordered content. Presence synchronization and chat summaries presented similar challenges. A view-level listener model alone was insufficient for the desired live responsiveness, particularly for the first moments after app open. As a result, the project moved toward broader coordination of real-time listeners for key data paths. Contact synchronization also required careful data management. Device contacts must be normalized and matched against registered cloud users, then reflected in the local model as communication relationships. Just as importantly, those contact-derived chats must be pruned when the underlying contact no longer exists. Nearby discovery required coordination between the two applications. Vibcom publishes discovery-relevant user state, while Vibconnect queries and filters that state. This arrangement keeps identity and communication centralized in Vibcom while allowing Vibconnect to focus on discovery logic.

6.3 Key Features of Vibcom

The most important implemented feature of Vibcom is persistent real-time one-to-one messaging. Users can authenticate, send messages, receive replies, reopen the application, and

retain access to prior conversation history. This differentiates the project from experimental accessibility prototypes that support only limited or temporary interaction. A second defining feature is multimodal text entry. Vibcom supports keyboard input, gesture-based braille-style entry, and drawing-based character entry. This reflects the project's broader commitment to flexible accessibility rather than dependence on a single preferred method. A third major feature is the accessibility feedback engine. The application can provide spoken confirmation, tone-based braille-style feedback, and visual output in a coordinated way. This allows users to navigate, interpret state, and consume message content without relying on a single output modality. Additional features include contact-derived chat creation, live presence status, highlighted focus navigation on the message home screen, and a polished authentication and onboarding flow that integrates accessibility concerns into the core user journey.

6.4 Key Features of Vibconnect

Vibconnect extends the ecosystem by providing nearby user discovery and accessible contact acquisition. Its main interface presents nearby candidates one at a time and supports left-swipe skip and right-swipe acceptance. This interaction model is intentionally simple and consistent with the broader design philosophy of the system. The application also supports a saved-contact view and filtering logic to avoid showing self, duplicate contacts, or previously skipped entries as active discovery candidates. These behaviors are essential to maintaining a practical discovery flow. Nearby exchange through Multipeer Connectivity is another key feature. This allows nearby devices to exchange number-linked identity information directly, creating a local connection path alongside Firestore-backed geographic discovery. Most importantly, Vibconnect is implemented as a companion rather than a disconnected application.

Accepted users feed into the broader communication environment, allowing the discovery workflow to result in practical messaging relationships within Vibcom.

6.5 Development Challenges

Several significant challenges emerged during development. One recurring challenge involved maintaining stable and correct state across multiple interconnected systems. Messaging, contacts, nearby discovery, accessibility feedback, and local persistence all interact, so changes in one subsystem often exposed regressions elsewhere. Another major challenge involved feedback timing and clarity. In an accessibility-first application, delays in mode switching, late feedback, or ambiguous confirmation are not minor annoyances. They directly affect orientation and user confidence. As a result, timing refinement became a substantive engineering concern rather than a purely aesthetic one. Gesture design also posed challenges. It was necessary to support a rich enough interaction set for messaging and discovery without creating a gesture language that was difficult to learn or easy to mis-trigger. This required repeated simplification and standardization of gesture behavior. Cross-application coordination introduced further complexity, especially around shared identity assumptions, contact propagation, and nearby exchange. Location-based discovery added another layer of difficulty because it depends on permissions, recent location state, and privacy-sensitive design decisions. Finally, a major development challenge was polish itself. In this project, polish includes timing, layout coherence, naming consistency, live state continuity, and feedback reliability. These are not superficial improvements; they are part of what makes the system genuinely usable and professionally presentable.

7. Evaluation

The evaluation of Vibcom and Vibconnect should be understood as a prototype-scale validation rather than a formal large-sample experimental study. The project did not collect extensive quantitative metrics or conduct a broad controlled user study. Instead, evaluation was based on structured functional testing, accessibility validation, cross-device testing, and iterative refinement informed by user feedback. Within that scope, the evaluation provides evidence of feasibility, technical maturity, and practical usability.

7.1 Functional Testing

Functional testing focused on correctness and consistency across the major workflows of both applications. In Vibcom, this included phone-number authentication, message persistence, real-time message propagation, presence updates, chat ordering, contact-based chat creation, and contact-based chat removal. In Vibconnect, it included self-exclusion, skip persistence, nearby candidate filtering, saved-contact management, nearby exchange, and integration of accepted users into the contact workflow. Messaging functionality was tested across multiple devices to verify that sent messages appear on recipient devices in a timely way and remain available after leaving and reopening the conversation. Earlier stages of development revealed issues such as delayed initial synchronization, incomplete live updating of open chats, and unstable conversation ordering. Subsequent architectural and state-management refinements improved these behaviors and resulted in a more stable final system. Contact integration was evaluated not only for positive creation behavior but also for correct removal behavior. The system was tested to ensure that when a new contact with a registered phone number is added, a chat becomes available, and when that contact is deleted, the corresponding contact-derived communication

pathway is removed. This is an important result because it confirms that the communication environment reflects the actual contact graph rather than drifting into stale state. Vibconnect was functionally tested for nearby discovery behavior, exclusion of self, prevention of duplicate presentations, persistence of skip decisions, nearby exchange behavior, and contact saving. Testing also examined whether accepted entries were correctly added to contacts and whether saved-state logic remained aligned with actual device contacts. Taken together, functional testing indicates that the final implemented system supports its major intended workflows reliably, though this reliability was achieved through substantial iterative correction of synchronization and state-management issues.

7.2 Accessibility Validation

Accessibility validation examined whether the system could be used meaningfully without vision and whether the accessibility features were integrated into the interaction model rather than merely attached to it. This involved testing complete task flows in Vibcom and Vibconnect through gesture input and nonvisual feedback. In Vibcom, validation included navigating the message home screen, opening a highlighted chat, composing messages through nonstandard input methods, switching interaction modes, and receiving confirmation through speech and tone-based output. In Vibconnect, validation included moving through the discovery stack, accepting or skipping candidates, opening nearby exchange, and interpreting discovery or error state through accessible feedback. One important observation from accessibility validation is that feature presence alone is insufficient. A feature may be technically accessible in principle, but if feedback timing is delayed, if the system is inconsistent across screens, or if visual controls remain necessary for completion, accessibility remains weak. This insight shaped multiple later

refinements in the system, particularly around gesture consistency, live updates, mode switching, and removal of visible-button dependence in some flows. Another important outcome is that multimodal feedback significantly improved accessibility quality. Spoken output, tone-based braille-style cueing, and visible state indicators each played different roles, and the system was strongest when these channels reinforced one another rather than operating in isolation.

7.3 Usability Assessment

Usability was assessed through iterative observation and user-informed refinement rather than through formal questionnaire instruments or large-scale experimental studies. Even within that qualitative scope, several patterns were clear. First, usability improved markedly as the interaction language became more unified. Reusing the same swipe logic across discovery and nearby exchange, aligning braille-mode controls across screens, and preserving stable structural patterns reduced the user's need to relearn controls and improved confidence. Second, interface polish was directly tied to usability. Inconsistent number formatting, awkward timing of live updates, mismatched toggle layouts, unstable status indications, or uneven scaling all made the applications feel less trustworthy and less professional. Resolving such issues improved not only the visual quality of the applications but also their practical interpretability. Third, continuity proved central to perceived quality. The system became significantly more convincing as messages persisted correctly, chats remained in sensible order, nearby discovery logic behaved consistently, and contact-derived state stayed aligned with actual device contacts. Overall, the usability assessment suggests that Vibcom and Vibconnect form a coherent and practically usable communication ecosystem. Their strongest contribution is not statistical performance evidence,

but the demonstration that accessibility-first messaging and nearby discovery can be made to work together in a stable, polished, and credible way.

8. Discussion

The project demonstrates that accessible mobile communication can be approached as a complete system rather than a narrow assistive prototype. Vibcom and Vibconnect are significant not only because they introduce alternative input and feedback methods, but because they integrate those methods into identity management, contact handling, live messaging, and nearby user discovery.

8.1 System Strengths

The most significant strength of the system is its accessibility-first design orientation. The applications were not built as ordinary mobile products with accessibility layered on later. Instead, their interaction model, navigation structure, and feedback logic were shaped around nonvisual use from the outset. This results in a more coherent experience than would typically emerge from retrofitting a conventional interface. A second major strength is breadth of functionality. Vibcom supports persistent real-time messaging, contact-based communication, multiple input methods, and multimodal feedback. Vibconnect extends the system by supporting nearby discovery and local nearby exchange. This makes the project substantially more comprehensive than many accessibility-oriented prototypes described in the literature. A third strength lies in the use of commodity hardware and mainstream infrastructure. By building on smartphones, tablets, Firebase services, and standard iOS frameworks, the system avoids dependence on specialized hardware and therefore points toward a more scalable and economically realistic accessibility model. A final strength is the project's emphasis on polish and consistency. Accessible applications are often discussed primarily in terms of functional accommodation, but this project demonstrates that professional presentation, clean interaction

logic, and stable real-time behavior are equally important to perceived quality and practical trustworthiness.

8.2 Design Tradeoffs and Challenges

The project also reveals important tradeoffs. One of the clearest is between deployability and tactile fidelity. By using consumer mobile hardware, the system becomes more accessible in cost and availability, but it cannot fully replicate the spatial immediacy of a dedicated refreshable braille display. Another tradeoff concerns gesture complexity. A richer gesture language could expose more direct control, but it would likely come at the cost of learnability and consistency. The project therefore favors a smaller and more stable gesture vocabulary, accepting some constraint in favor of clarity. A further tradeoff emerges in the hybrid local-cloud architecture. Local persistence improves continuity and responsiveness, while cloud synchronization supports real-time multi-device behavior. However, combining the two introduces substantial engineering complexity around merge logic, listener timing, and state reconciliation. The project also highlights the fact that accessibility is not only a user-interface challenge. It is a systems challenge involving timing, persistence, backend coordination, permission handling, and inter-application consistency. This is one of the more important lessons of the work.

8.3 Implications for Accessible Application Design

This project has several implications for accessible application design more broadly. First, it supports the argument that accessibility should shape software architecture from the beginning. If accessibility is treated as a foundational constraint, the resulting interaction and system logic are more coherent than if accessibility is added afterward as a compliance layer.

Second, it demonstrates the value of multimodal interaction. No single feedback or input method is sufficient for all users or all contexts. Systems that combine multiple accessible pathways can better support real-world use. Third, the project suggests that communication accessibility should be understood as an ecosystem problem rather than only a messaging problem. Messaging in practice depends on identity, contacts, discovery, and state continuity. Vibcom and Vibconnect together reflect this broader framing. Finally, the system reinforces that accessible design and software quality are closely linked. Stability, consistency, timing, formatting, and polish are not secondary to accessibility; they are part of it.

9. Limitations

Despite the strengths of the project, several limitations remain. The first limitation is hardware-level. Consumer smartphones and tablets do not provide the spatial tactile resolution of dedicated braille hardware. As a result, Vibcom's braille-inspired feedback cannot be considered a direct replacement for a refreshable braille display. The second limitation concerns evaluation scope. The project includes functional testing, accessibility validation, and user-informed refinement, but it does not include a large formal study with a broad population of blind or deafblind participants. Accordingly, the conclusions of this thesis should be interpreted as evidence of feasibility and design maturity rather than broad statistical validation. A third limitation is platform dependence. The system is tightly coupled to iOS frameworks and Apple-specific device capabilities. This enables strong native integration, but limits portability beyond the Apple ecosystem. A fourth limitation is that several features depend on permissions and live system state. Nearby discovery depends on location permission and recent discovery metadata. Nearby exchange depends on local peer conditions. Contact-derived chat behavior depends on access to contacts. These dependencies are reasonable for the system's goals, but they do introduce practical constraints. Finally, the project intentionally focuses on core communication and discovery rather than attempting to replicate the full feature set of large-scale mainstream messaging platforms. Group messaging, multimedia workflows, content moderation, and more advanced social matching logic remain outside the present scope.

10. Conclusion

This thesis presented Vibcom and Vibconnect as an integrated accessibility-first mobile communication system. Vibcom was developed as the primary messaging platform, supporting persistent real-time communication, multiple text-entry methods, contact-derived conversations, and multimodal feedback. Vibconnect extends this environment by enabling nearby discovery and accessible contact acquisition through a gesture-centered, nonvisual interaction model. The project demonstrates that accessible mobile communication can be designed as a complete ecosystem rather than as a collection of isolated assistive features. By integrating authentication, persistence, cloud synchronization, nearby discovery, and multimodal interaction, the system shows that accessibility can shape not only the visible interface but also the underlying software architecture. The work also highlights an important lesson: accessibility is inseparable from system quality. Timing, state continuity, consistency, live update behavior, and overall polish all directly influence whether a nonvisual interface feels usable and trustworthy. In this sense, the contribution of the project is both technical and design-oriented. Although the current implementation has limitations, particularly in tactile fidelity and evaluation scale, it establishes a credible foundation for future research and development. Further work could include broader user evaluation, refinement of braille-style interaction, stronger backend policies, expanded communication features, and more advanced nearby discovery logic. Even in its present form, however, Vibcom and Vibconnect demonstrate that accessible messaging and discovery can be implemented in a way that is technically meaningful, academically defensible, and practically relevant.

11. References

Dolphin, S., Downing, M., Cirrincione, M., Samuta, A., Leite, K., Noble, K., & Walsh, B. (2024). Information Accessibility in the Form of Braille. IEEE open journal of engineering in medicine and biology, 5, 205–209. https://doi.org/10.1109/OJEMB.2024.3364065.

Jayant, C., Acuario, C., Johnson, W., Hollier, J., & Ladner, R. (2010). V-Braille: Haptic Braille perception using a touch-screen and vibration on mobile phones. In Proceedings of the 12th International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '10) (pp. 295–296). Association for Computing Machinery. https://doi.org/10.1145/1878803.1878878.

Khusro, S., Shah, B., Khan, I., & Rahman, S. (2022). Haptic feedback to assist blind people in indoor environment using vibration patterns. Sensors, 22(1), 361. https://doi.org/10.3390/s22010361.

Martinez, J., & Kamei-Hannan, C. (2025). Preferences and opinions of braille users on the development of a multiline braille display. Journal of Blindness Innovation and Research, 15(1). https://nfb.org/images/nfb/publications/jbir/jbir25/jbir150101.html.

Muntean, M., & Militaru, F. (2022). Design science research framework for performance analysis using machine learning techniques. *Electronics, 11*(16), 2504. https://doi.org/10.3390/electronics11162504.

Nunamaker, J. F., Chen, M., & Purdin, T. D. M. (1990). Systems development in information systems research. *Journal of Management Information Systems, 7*(3), 89–106. http://www.jstor.org/stable/40397957.

Ozioko, O., Karipoth, P., Hersh, M., & Dahiya, R. (2020). Wearable assistive tactile communication interface based on integrated touch sensors and actuators. *IEEE Transactions on Neural Systems and Rehabilitation Engineering, 28*(6), 1344–1352. https://doi.org/10.1109/TNSRE.2020.2986222.

Pawluk, D. T., Adams, R. J., & Kitada, R. (2015). Designing haptic assistive technology for individuals who are blind or visually impaired. IEEE Transactions on Haptics, 8(3), 258–278. https://doi.org/10.1109/TOH.2015.2471300.

Rantala, J., et al. (2009). Methods for presenting Braille characters on a mobile device with a touchscreen and tactile feedback. IEEE Transactions on Haptics, 2(1), 28–39. https://doi.org/10.1109/TOH.2009.3.

Schroeder, F. K. (1996). Perceptions of Braille usage by legally blind adults. Journal of Visual Impairment & Blindness, 90(3), 210–218. https://doi.org/10.1177/0145482X9609000310.

Silverman, A. M., & Bell, E. C. (2018). The association between braille reading history and well-being for blind adults. Journal of Blindness Innovation and Research, 8(1). https://nfb.org/images/nfb/publications/jbir/jbir18/jbir080103.html.

Thomas, A., & Rufus, E. (2016). Alternate braille display designs: A review. Technology and Disability, 28(4), 123–132. https://doi.org/10.3233/TAD-160451.

Venable, J. R., Pries-Heje, J., & Baskerville, R. L. (2017). *Choosing a design science research methodology*. In *Proceedings of the 28th Australasian Conference on Information Systems (ACIS)* (Paper 112). Australasian Conference on Information Systems. https://aisel.aisnet.org/acis2017/112.

Voutsakelis, G., Tzimos, N., Kokkonis, G., & Kontogiannis, S. (2024). A haptic Braille keyboard layout for smartphone applications. Electronics, 13(12), 2408. https://doi.org/10.3390/electronics13122408.

12. Appendices

This section provides supplementary visual material that supports the system design, implementation, and evaluation presented in the main body of the thesis. The appendices include architectural diagrams, workflow diagrams, and representative user interface screenshots for Vibcom and Vibconnect. These materials are intended to illustrate the structure of the system, the relationship between its components, and the accessibility-centered interaction flows discussed throughout the paper.

12.1 System and Workflow Diagrams

Figure 1. High-level architecture of Vibcom and Vibconnect.

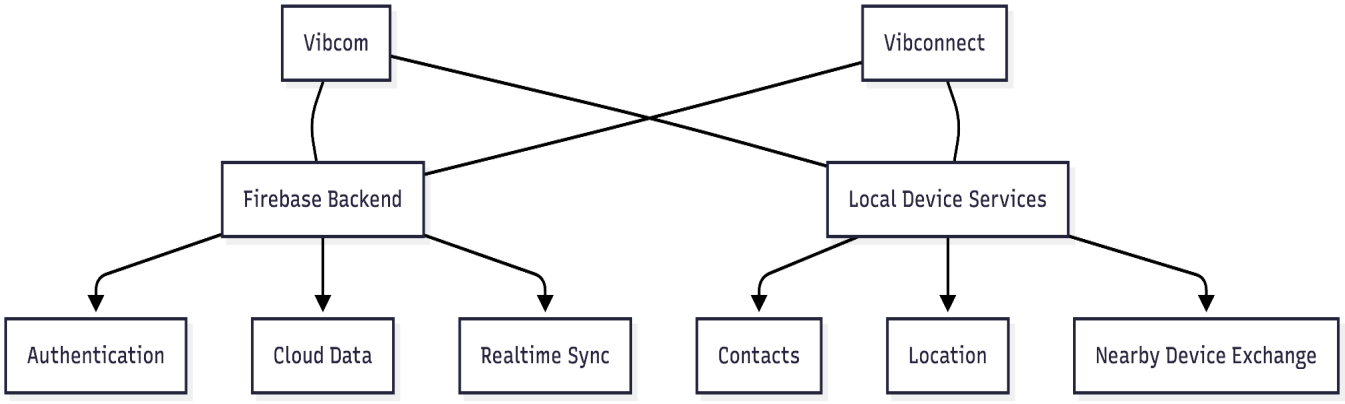


Figure 2. Vibcom messaging and synchronization architecture.

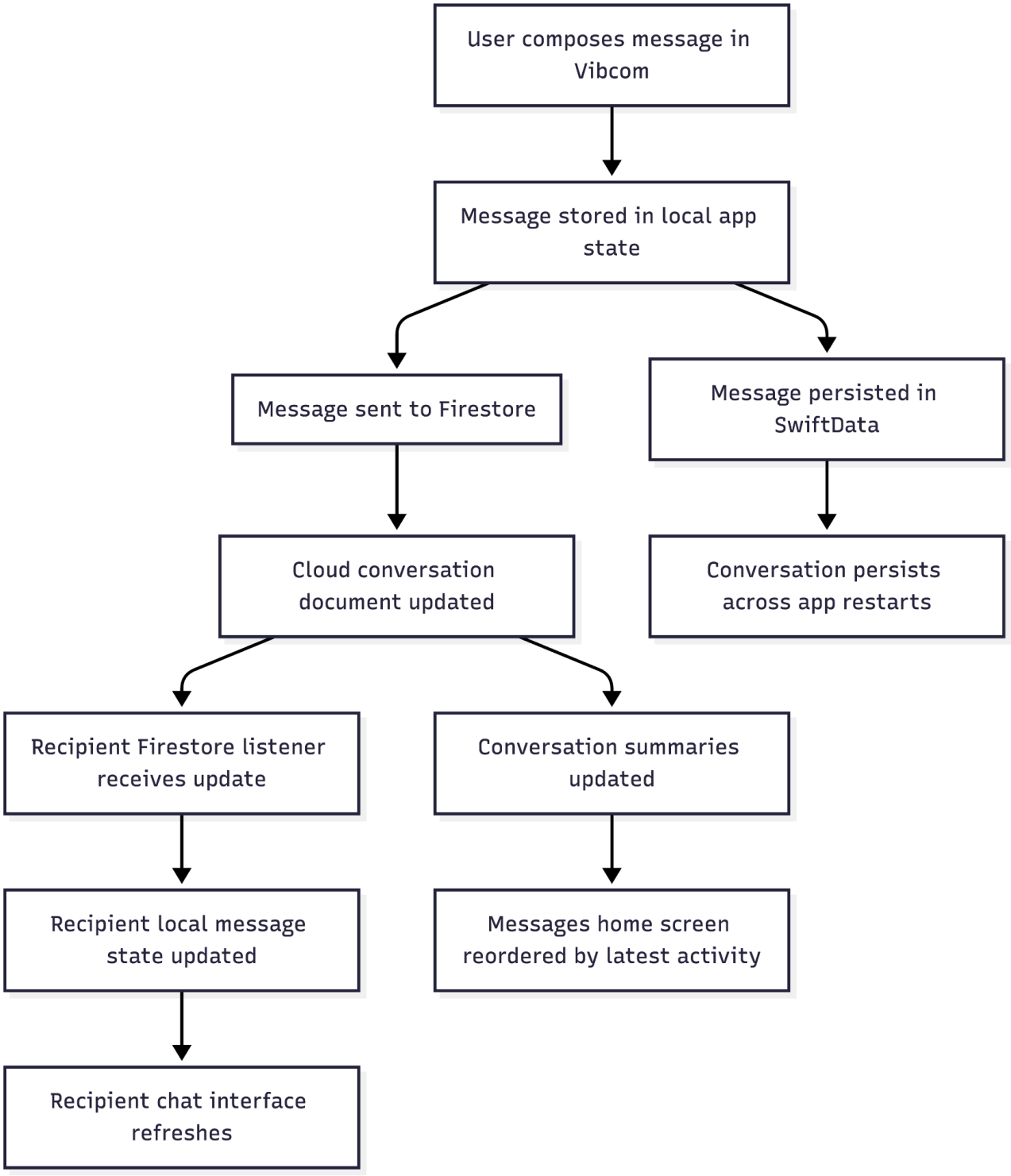


Figure 3. Vibcom contact synchronization workflow.

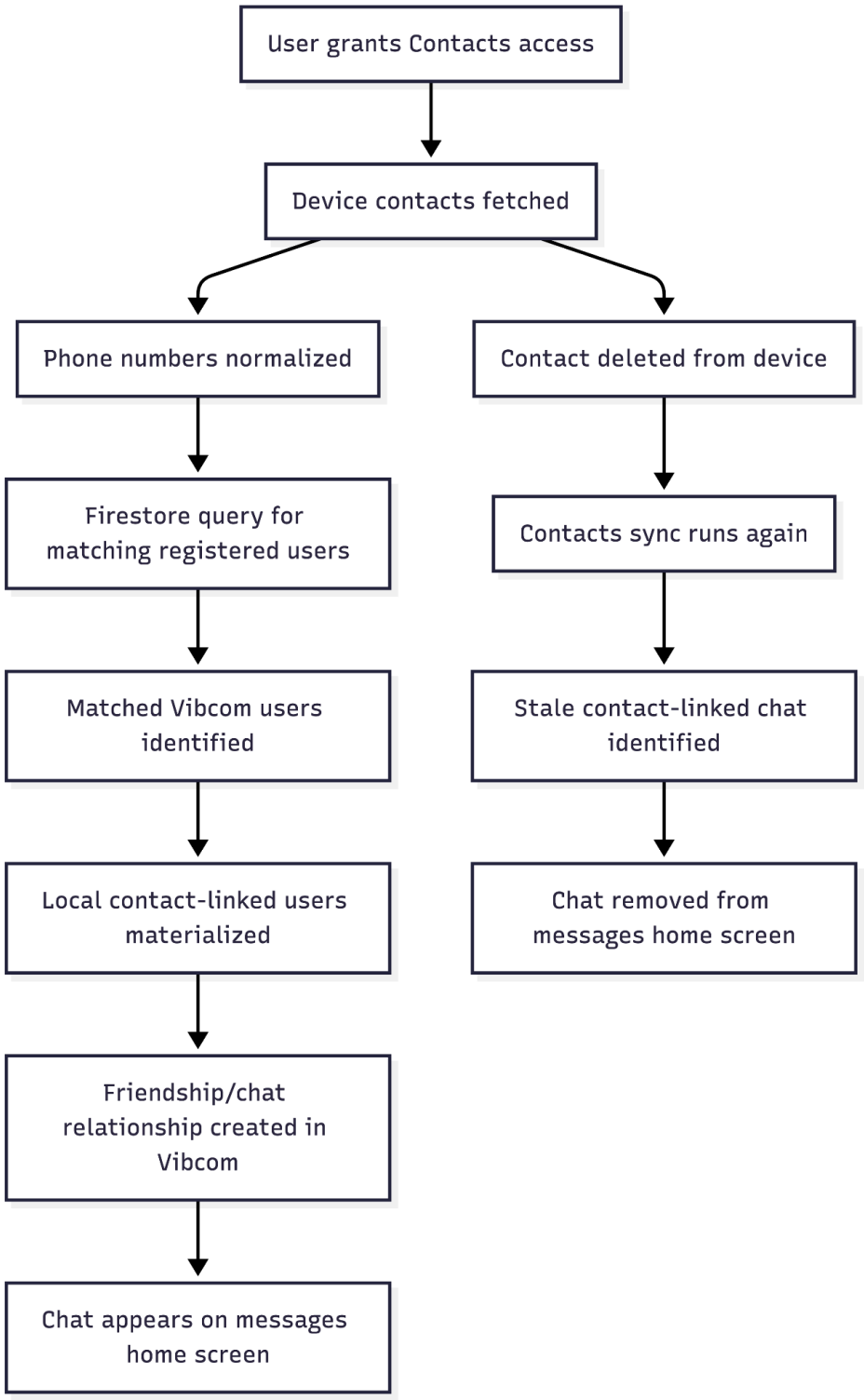


Figure 4. Vibconnect nearby discovery workflow.

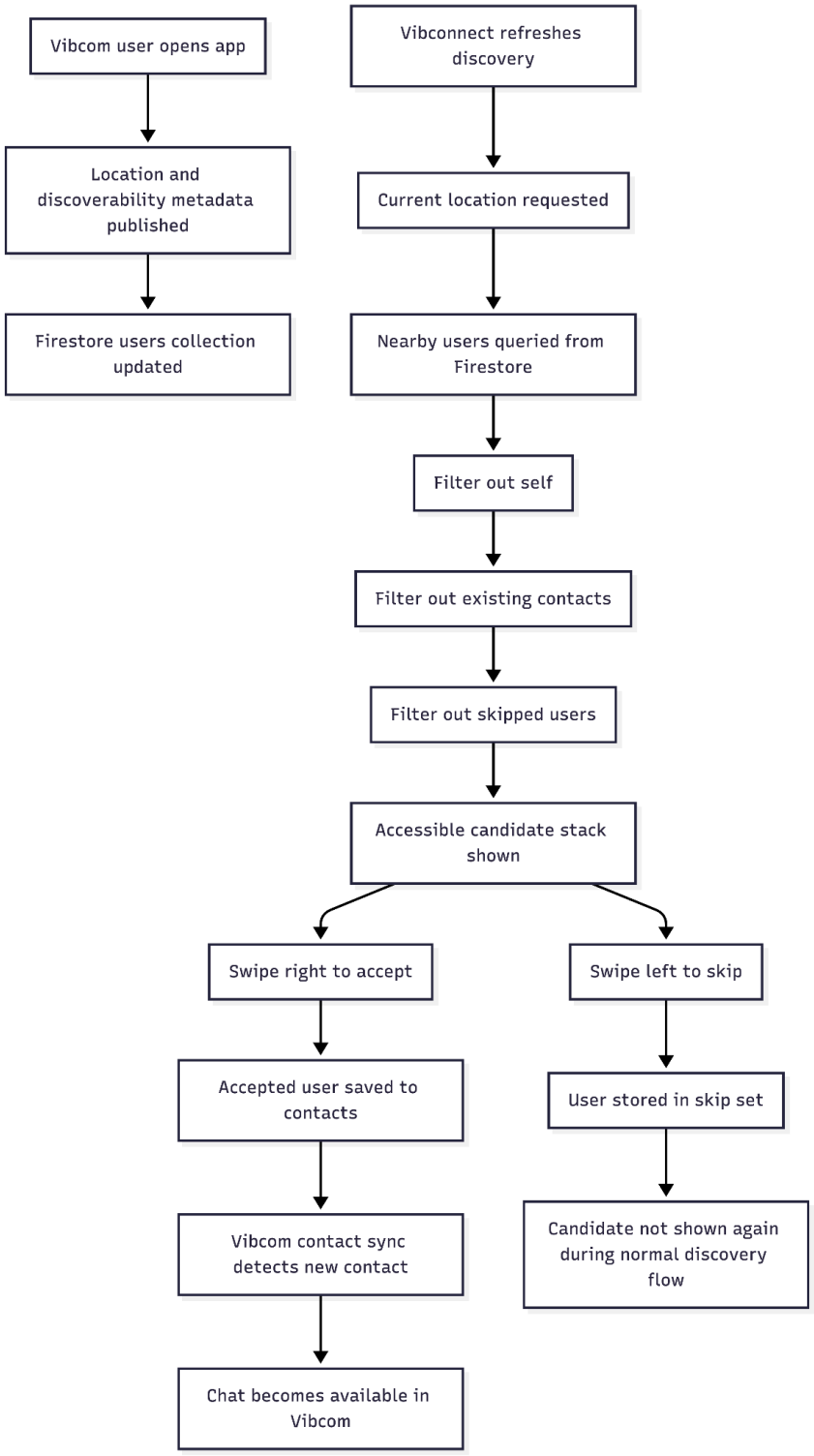


Figure 5. Nearby exchange workflow.

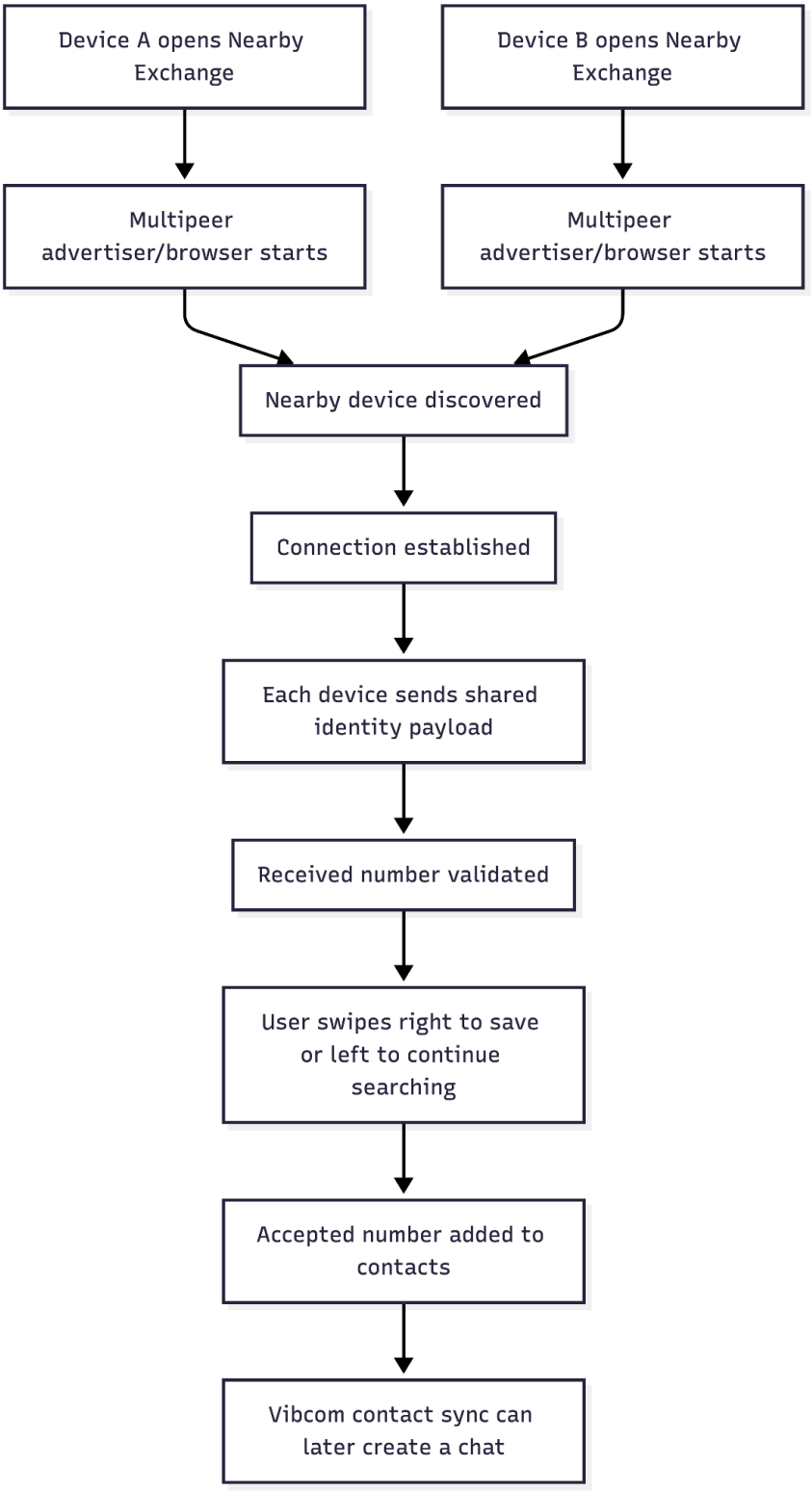


Figure 6. Accessibility interaction model across the system.

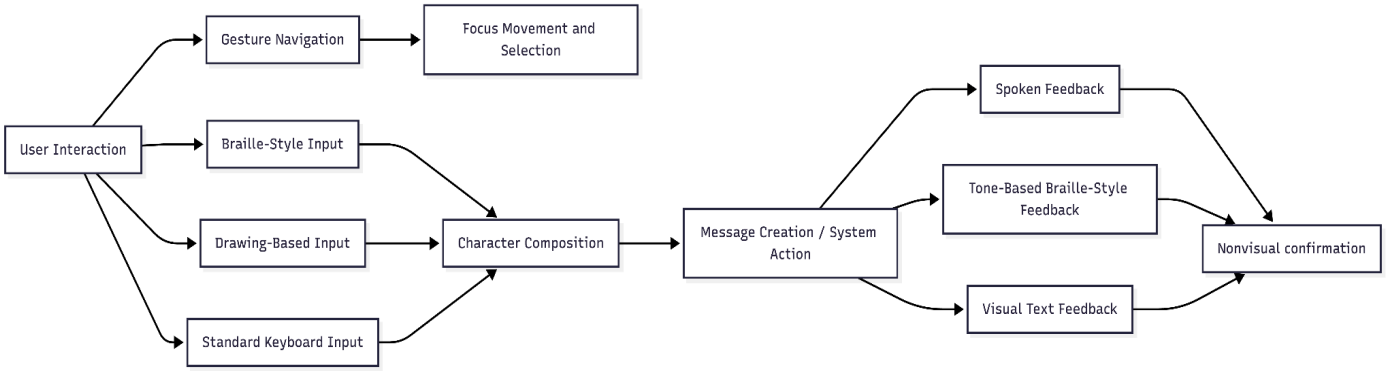


Figure 7. Vibcom authentication and onboarding flow.

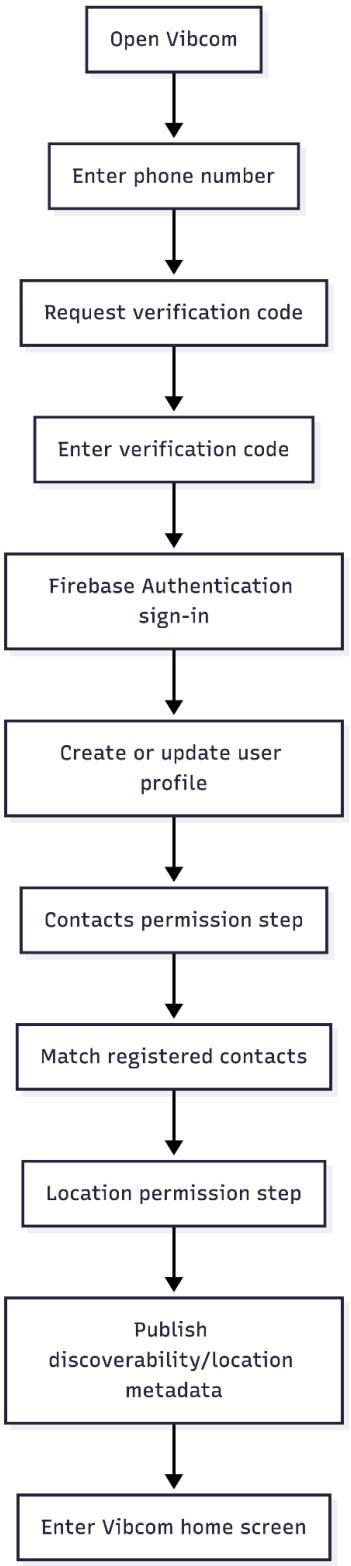
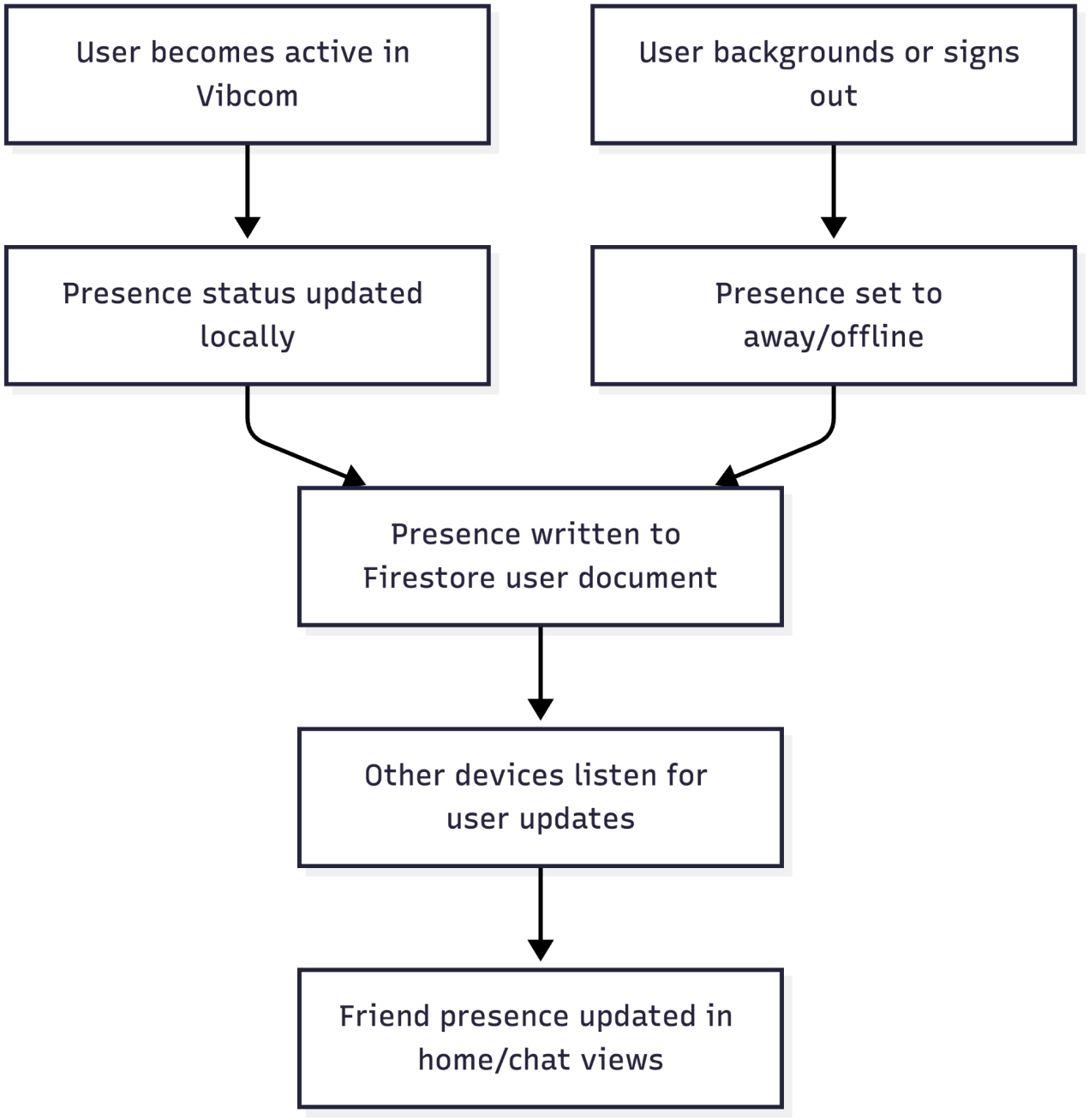


Figure 8. Real-time presence and status update flow.



12.2 User Interface Screenshots

Figure 9. Vibcom phone verification interface showing the initial authentication screen with braille mode enabled and gesture-based input available.

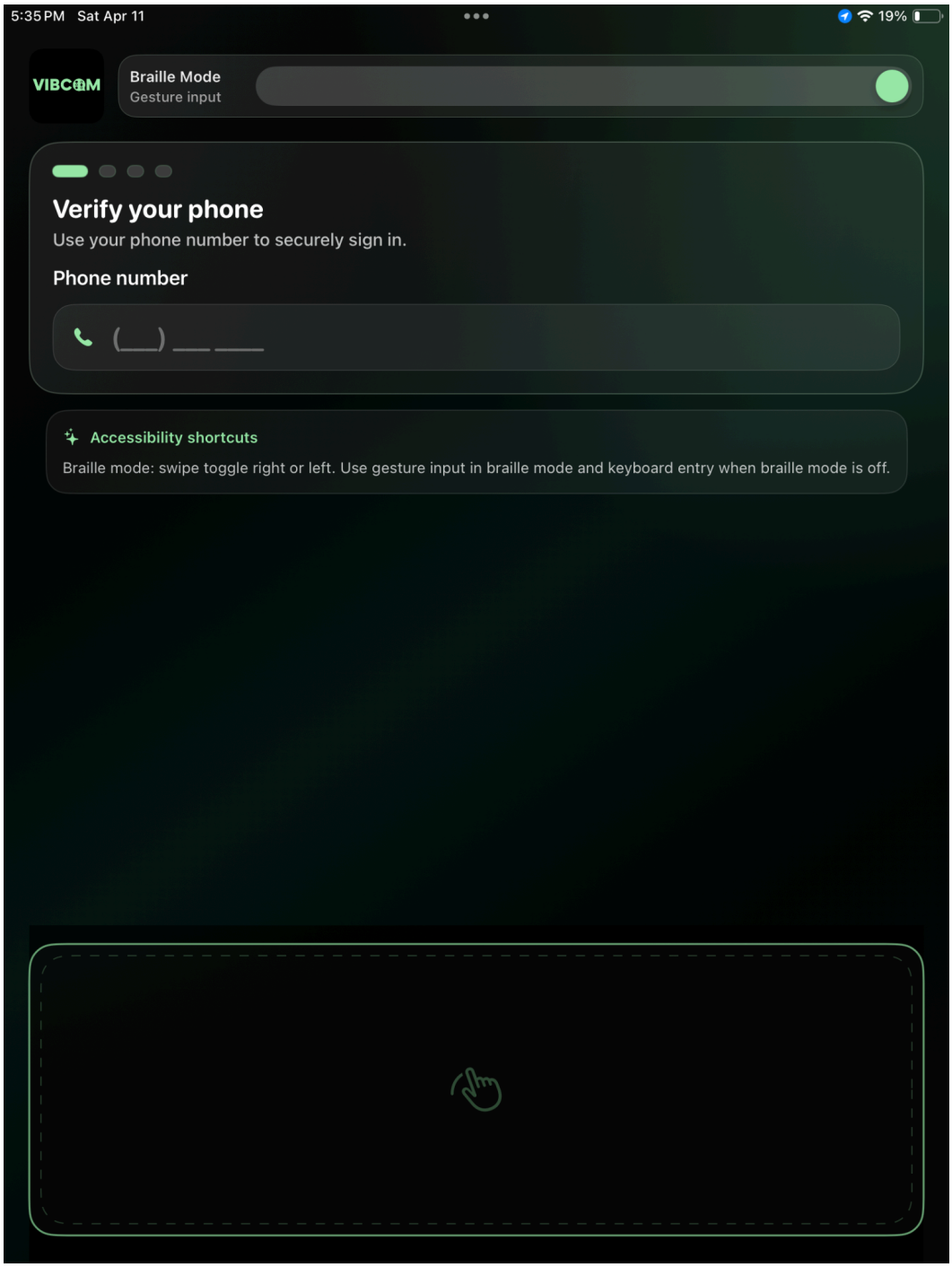


Figure 10. Vibcom verification code screen showing the second stage of phone-number authentication within the accessible onboarding flow.

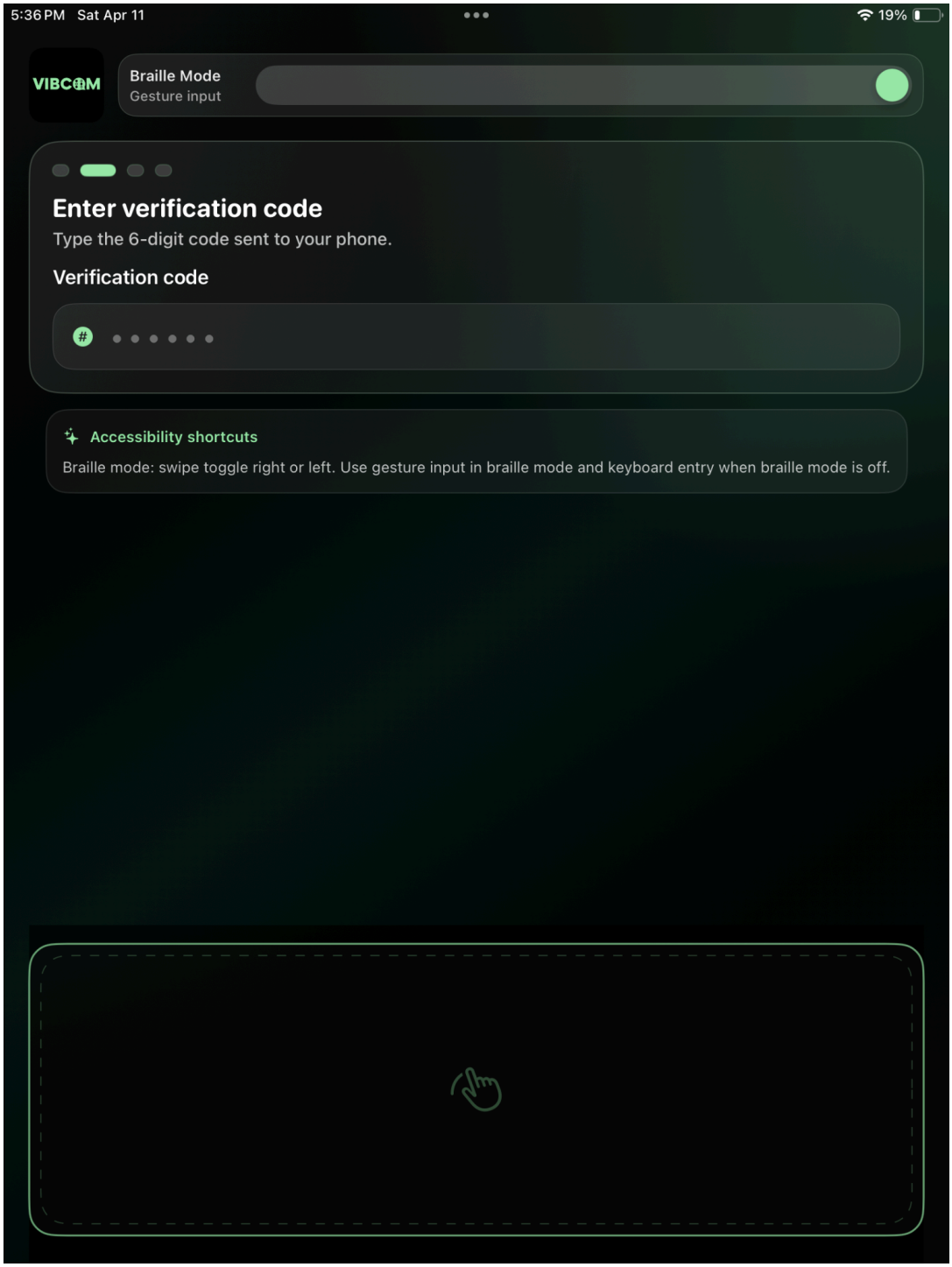


Figure 11. Vibcom contacts permission step, illustrating the integration of device contacts into the onboarding sequence for contact-based chat generation.

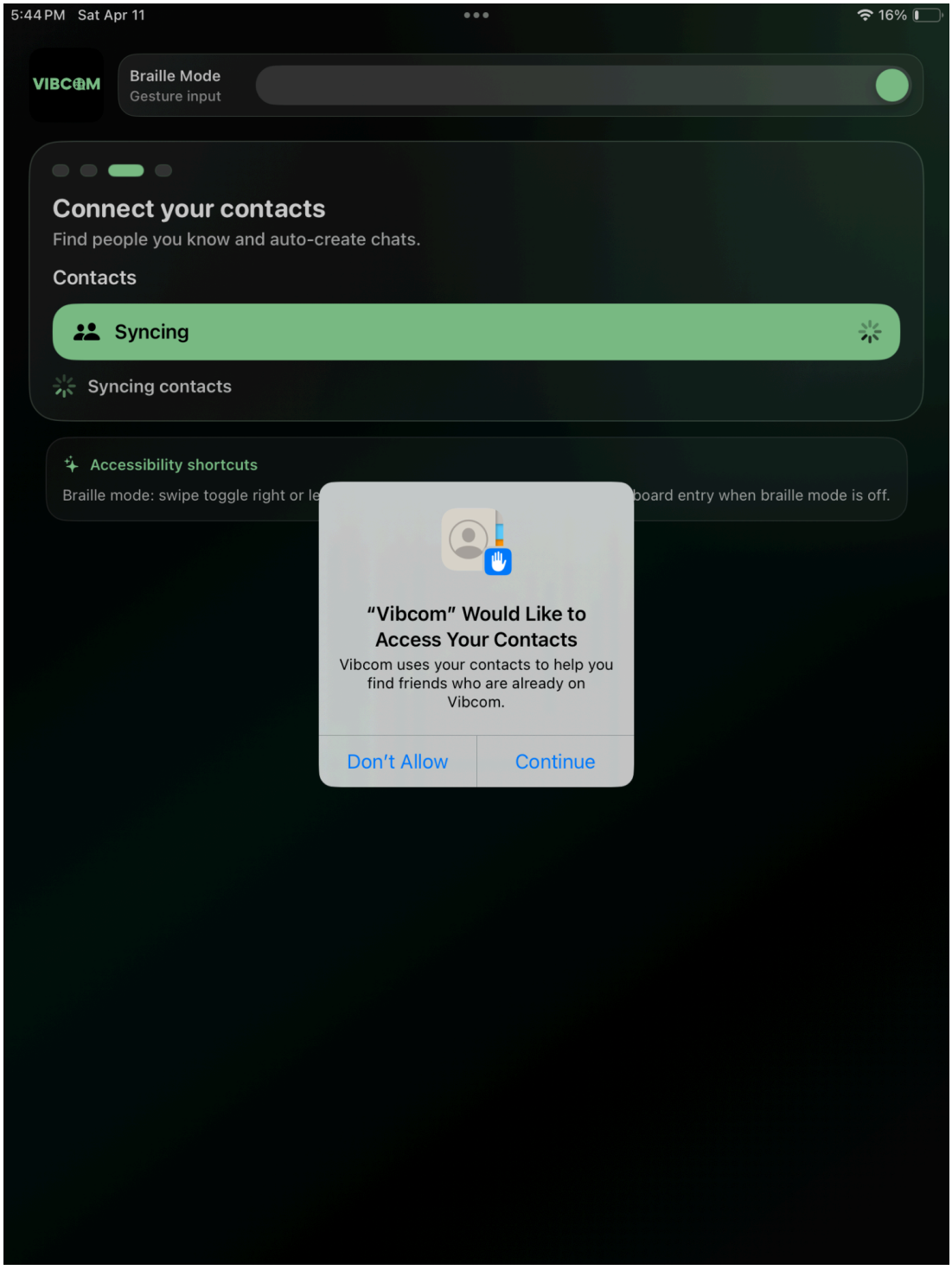


Figure 12. Vibcom location permission step, showing how nearby-discovery readiness is incorporated into the onboarding flow.

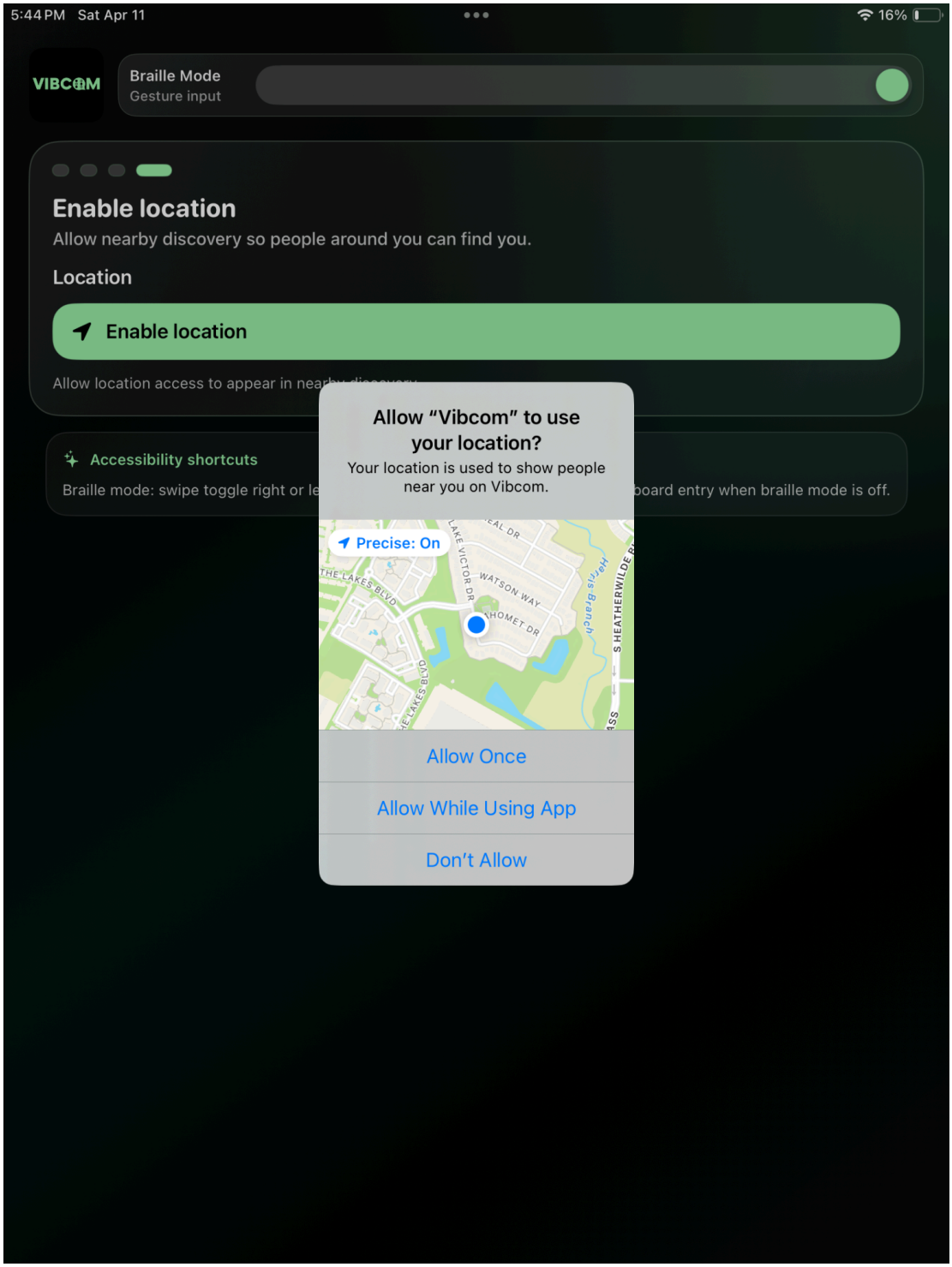


Figure 13. Vibcom messages home screen in the empty-state condition, showing focused navigation and the sign-out action when no chats are available.

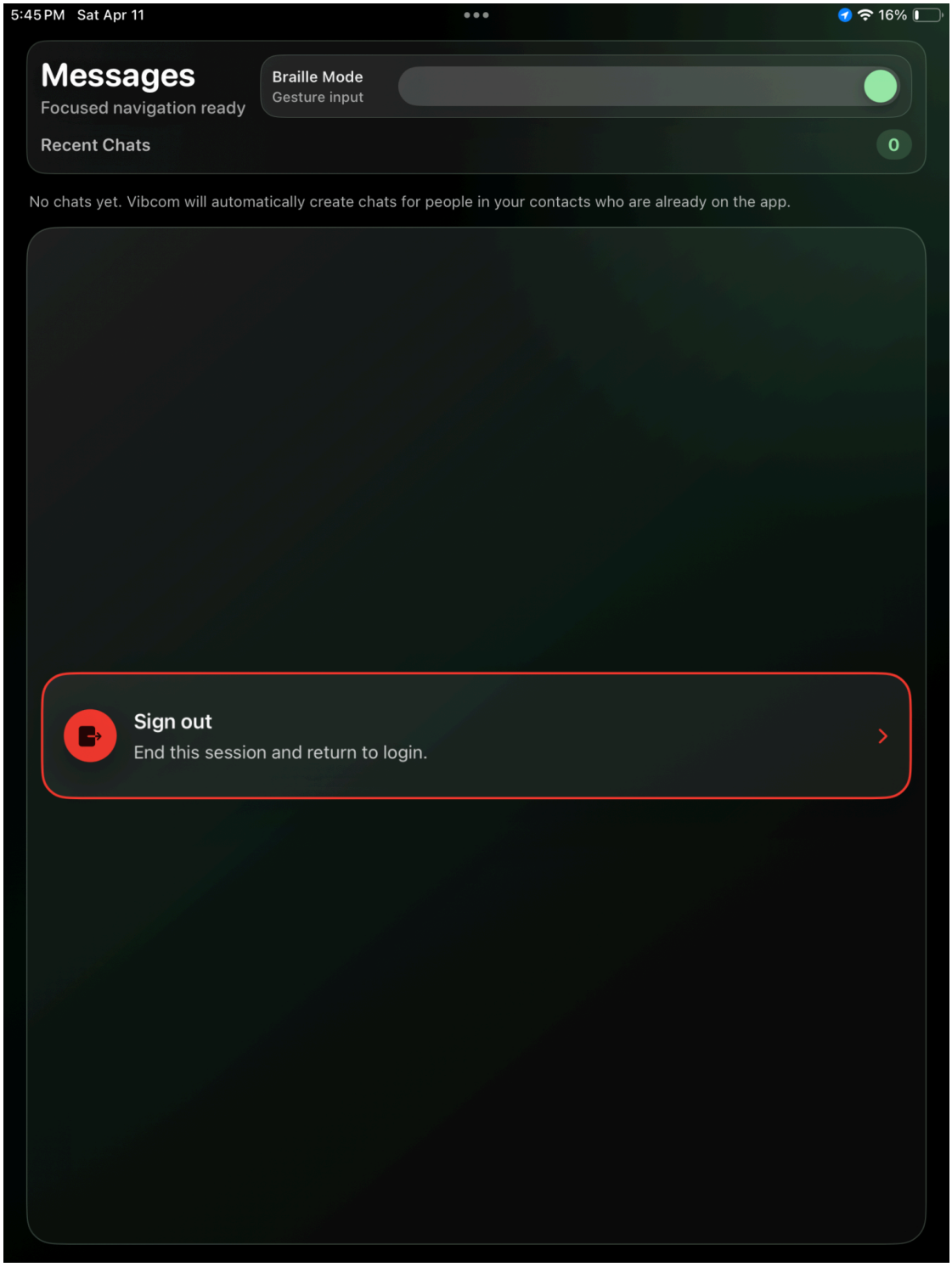


Figure 14. Vibcom messages home screen after contact synchronization, showing a contact-linked chat available for selection.

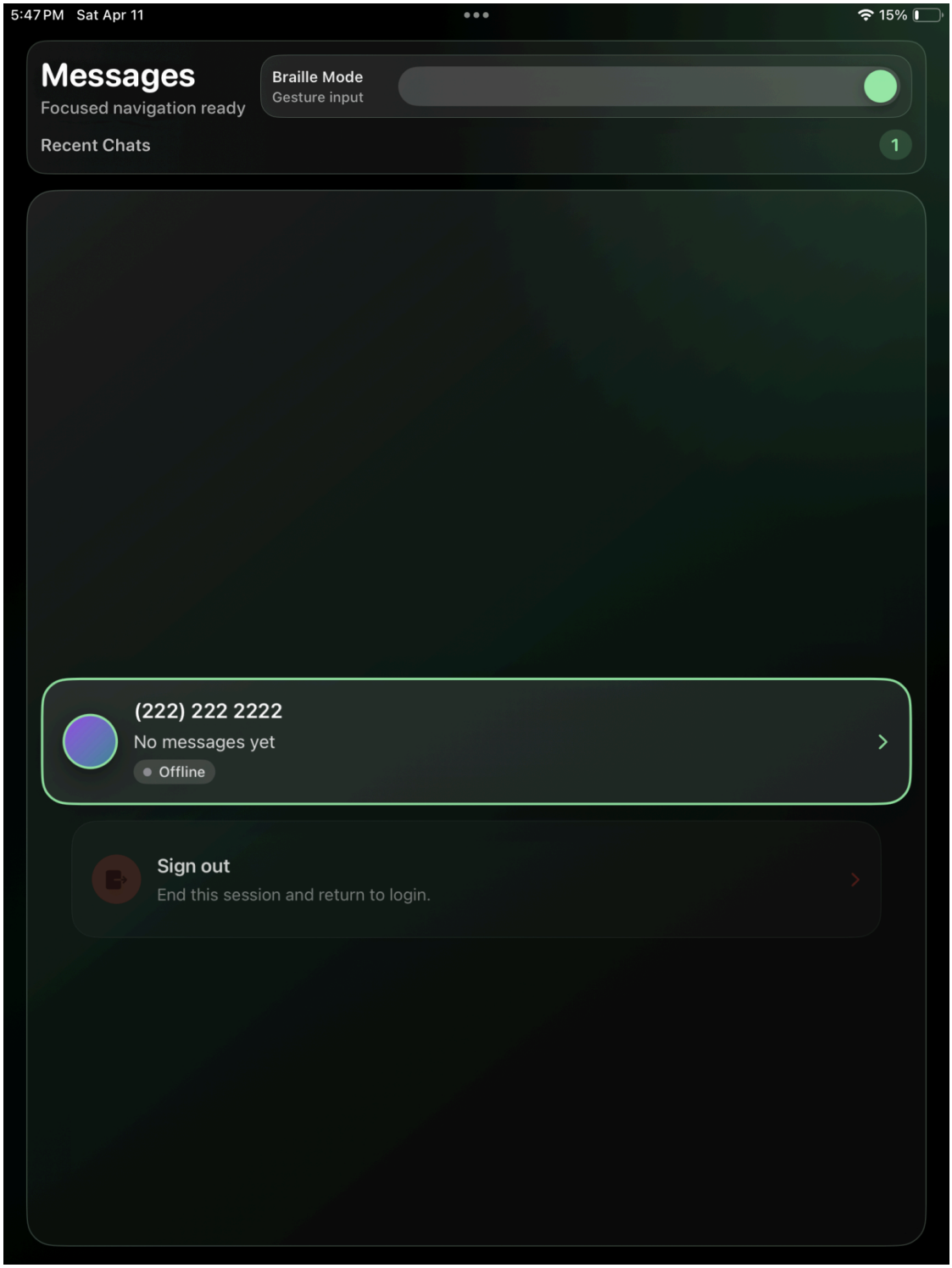


Figure 15. Vibcom chat detail interface showing a newly composed outgoing message and the gesture-based input area.

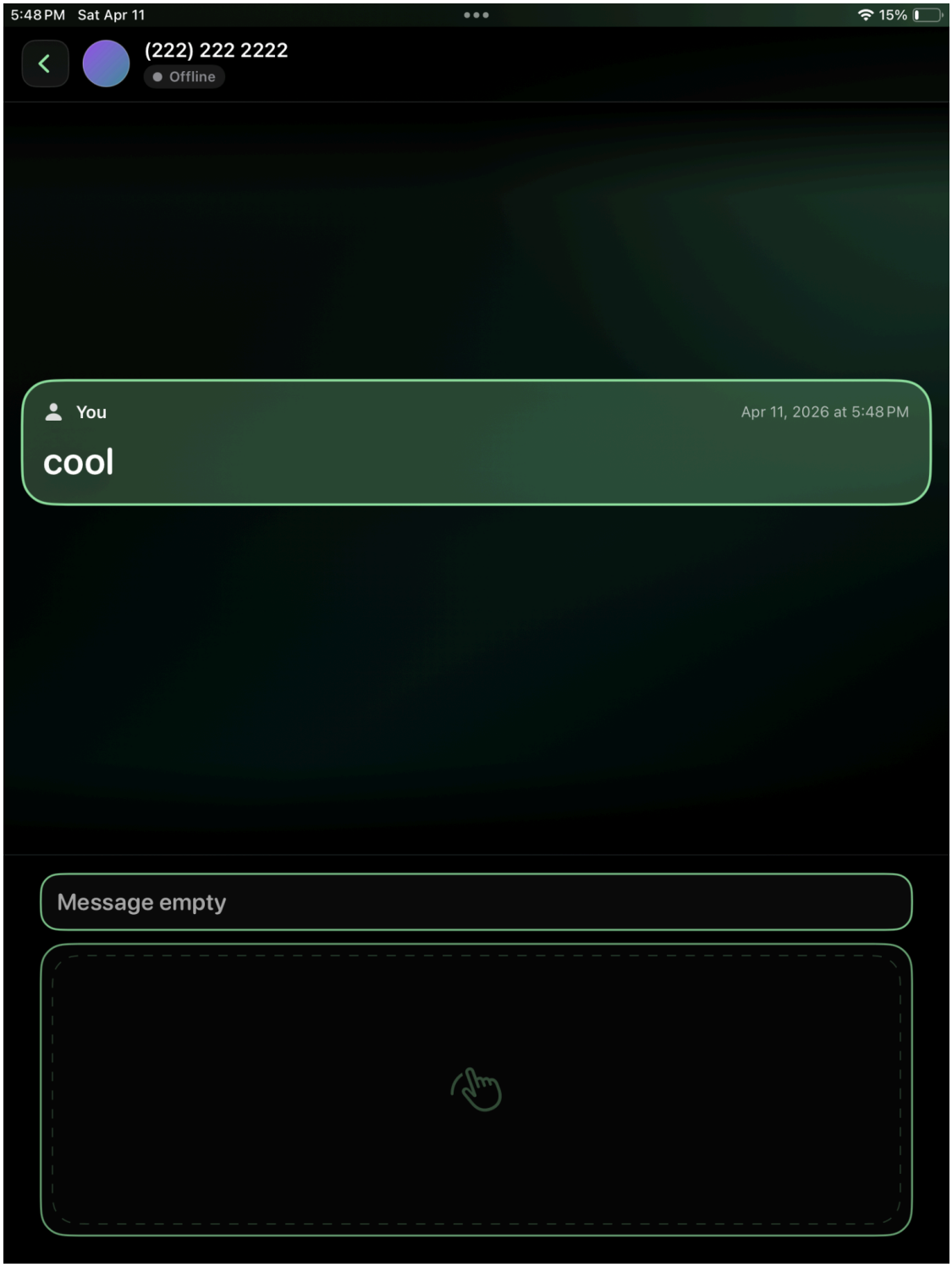


Figure 16. Vibcom chat detail interface showing a two-way conversation with live message updates and active presence indication.

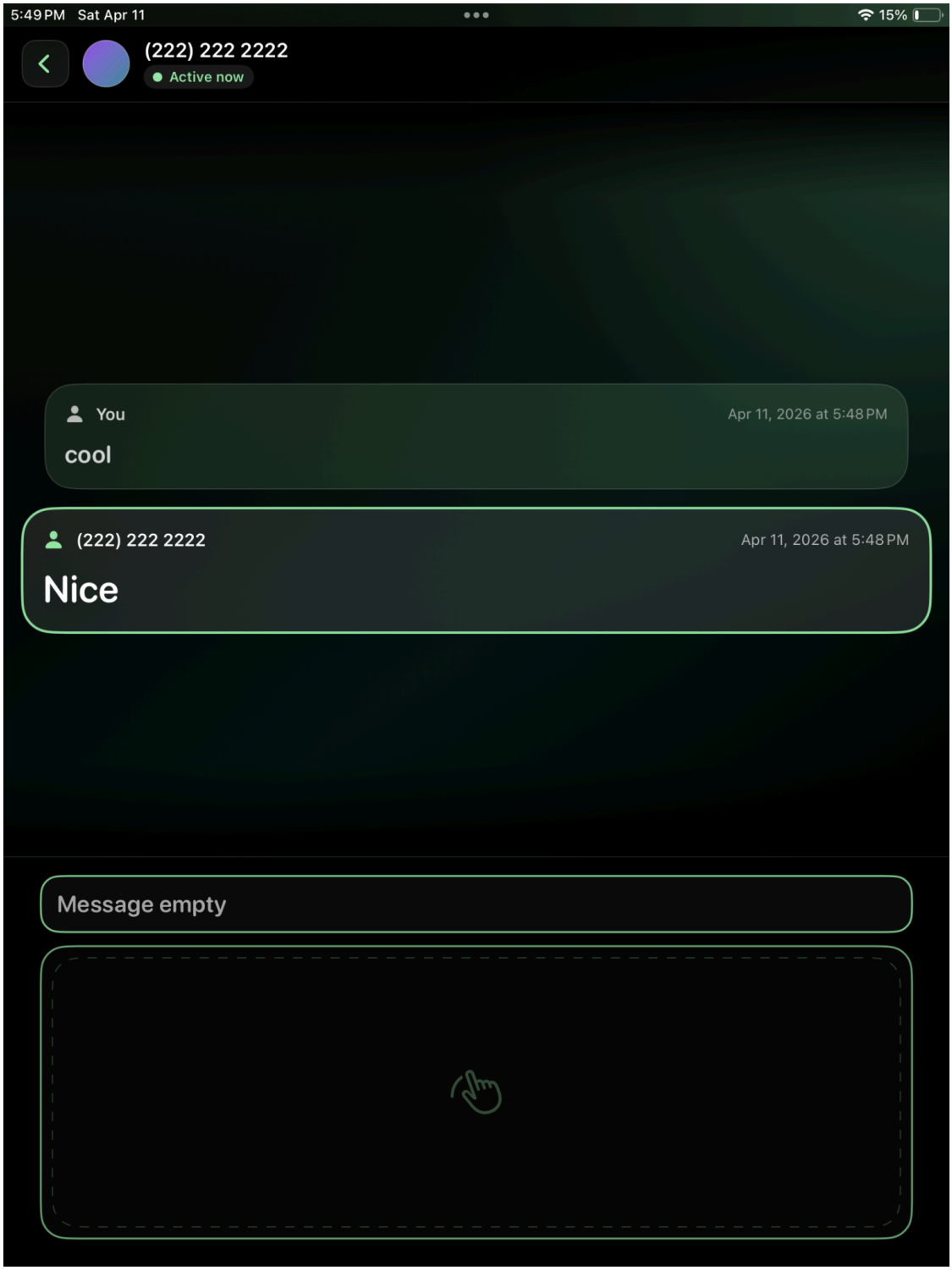


Figure 17. Vibconnect main discovery interface showing the nearby user card and the gesture-driven accept or skip interaction model.

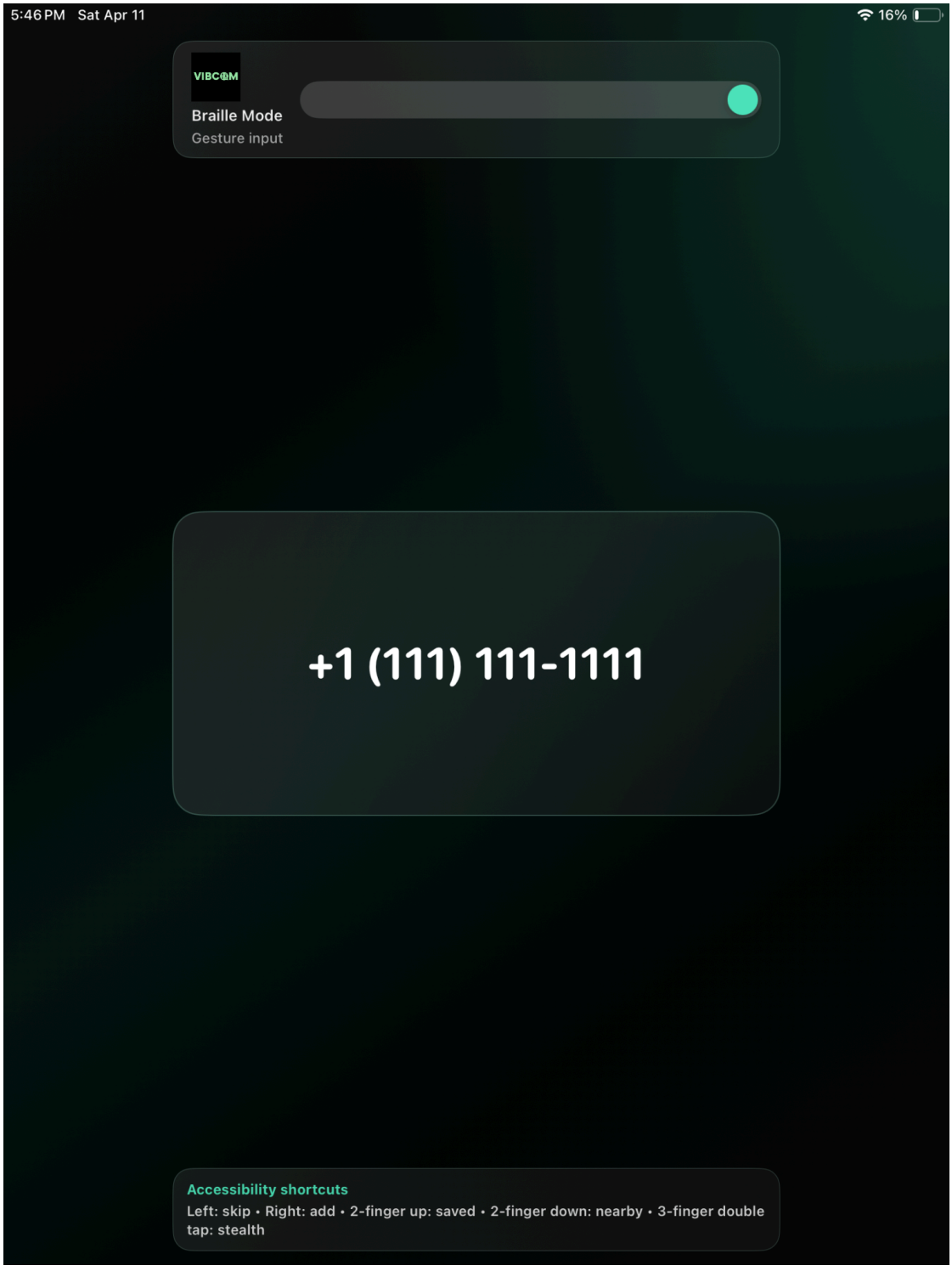


Figure 18. Vibconnect nearby exchange screen in the active searching state, illustrating local device discovery for direct number exchange.

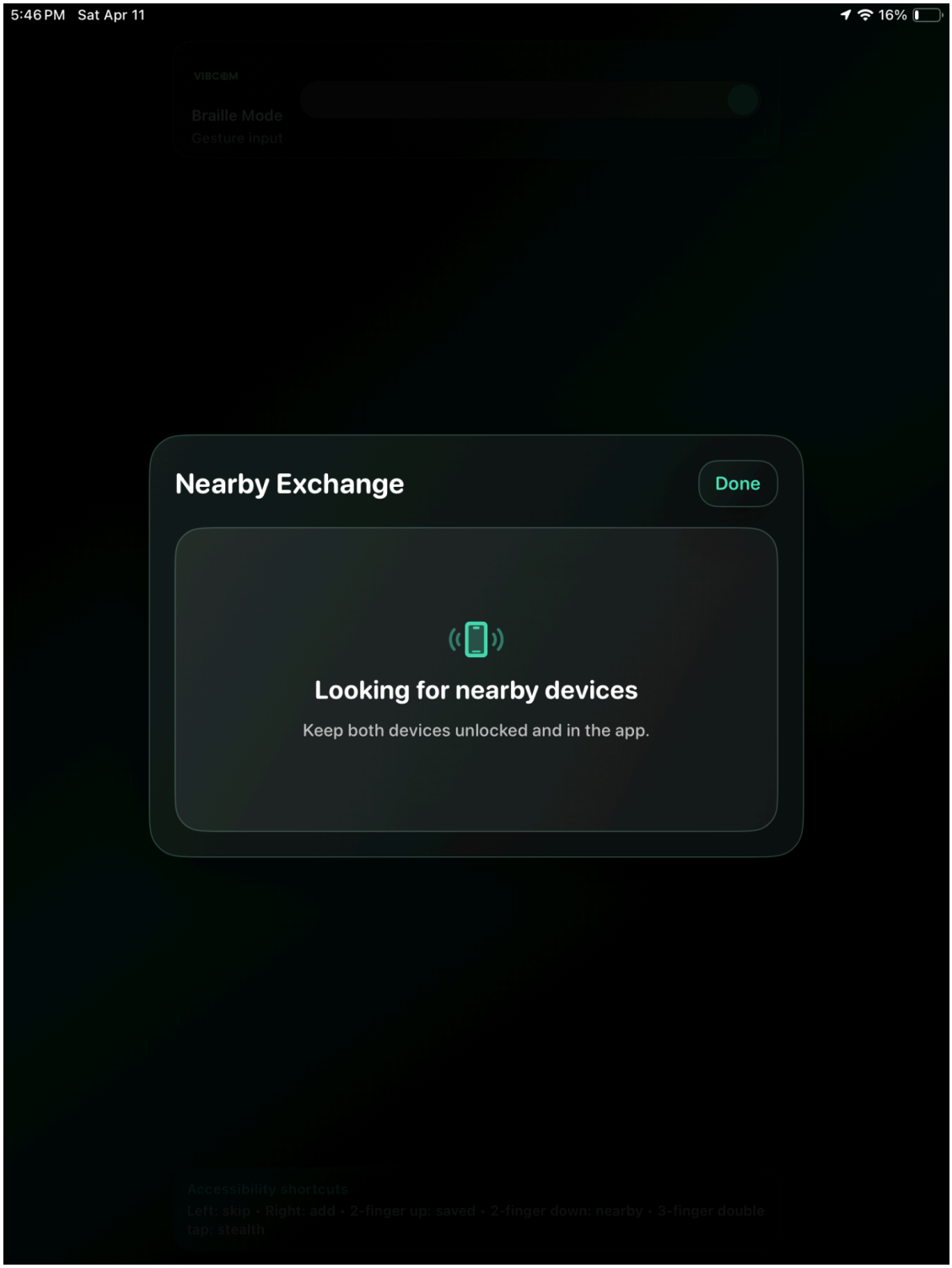


Figure 19. Vibconnect nearby exchange screen after a nearby number has been detected and prepared for acceptance through gesture interaction.

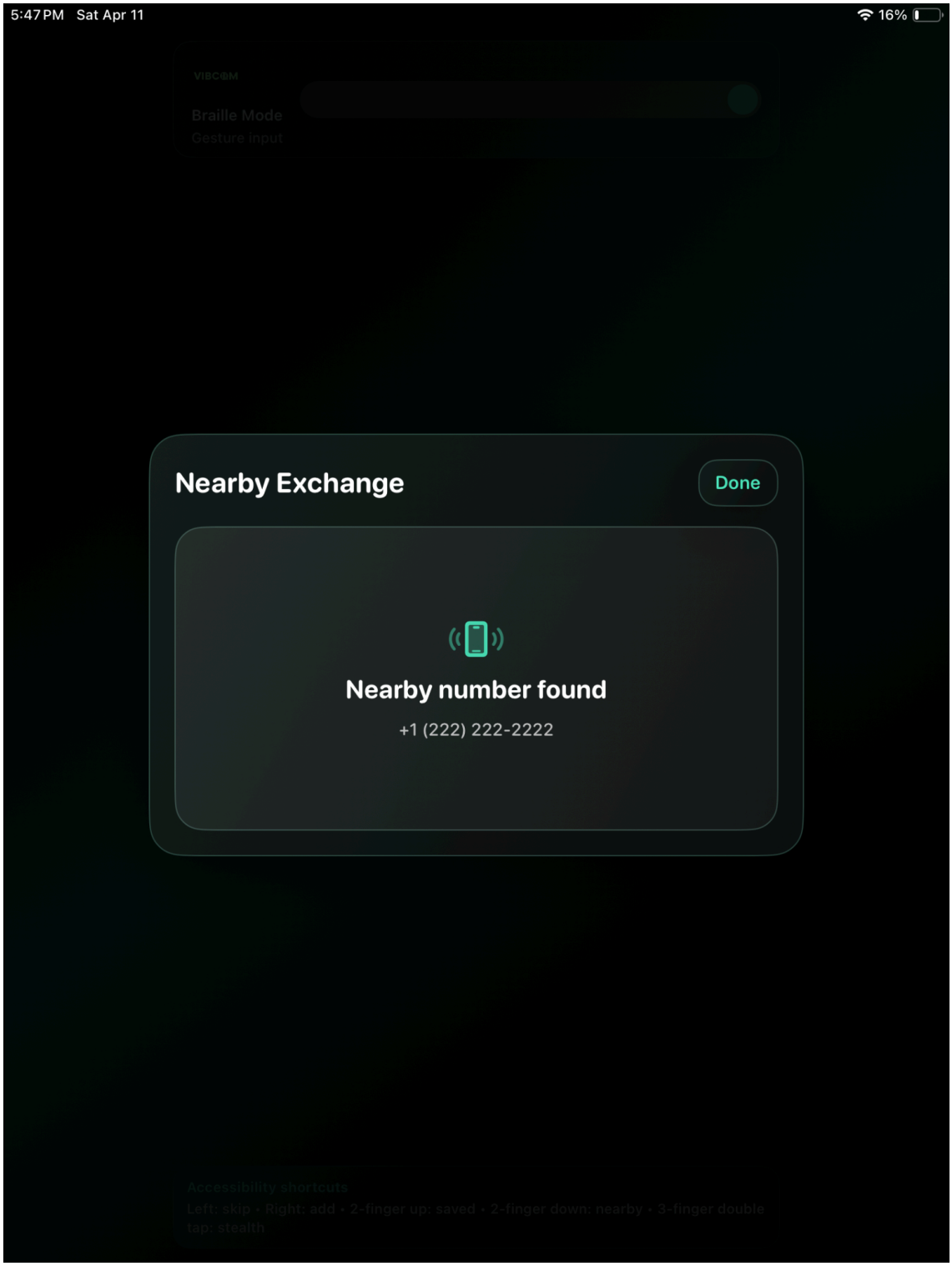


Figure 20. Vibconnect saved contacts screen showing a user accepted through the nearby discovery workflow.